

\$author-display-name

09/04/2026



DPUCZDX8G for Zynq UltraScale+ MPSoCs Product Guide (PG338)

\$author-display-name

Introduction

Features

IP Facts

Overview

Core Overview

Hardware Architecture

Development Tools

MPSoC Integration Example

Vitis AI Development Kit

Product Specification

Port Descriptions

Register Space

Interrupts

DPU Configuration

Introduction

Configuration Options

Advanced Tab

Summary Tab

DPUCZDX8G Peak Performance

Xilinx Model Zoo Performance

I/O Bandwidth Requirements

Clocking

Reset

Development Flow

Customizing and Generating the Core in the Zynq UltraScale+ MPSoC

Customizing and Generating the Core in the Vitis IDE

Example Design

Introduction

Vivado DPU TRD Flow

Vitis DPU TRD Flow

Additional Resources and Legal Notices

Xilinx Resources

Documentation Navigator and Design Hubs

References

Revision History

Please Read: Important Legal Notices

DPUCZDX8G for Zynq UltraScale+ MPSoCs Product Guide (PG338)

Introduction

The DPUCZDX8G is the Deep learning Processor Unit (DPU) designed for the Zynq® UltraScale+™ MPSoC. It is a configurable computation engine optimized for convolutional neural networks. The degree of parallelism utilized in the engine is a design parameter and can be selected according to the target device and application. At a high-level, the DPU is a microcoded compute engine which has an efficient, optimized instruction set, and which can support inference of most convolutional neural networks.

Features

- Supports one AXI slave interface for accessing configuration and status registers.
- Supports one AXI master interface for instruction fetch.
- Supports individual configuration of each channel.
- IP is available in multiple variants, scaling both in terms of logic resource utilization and parallelism. Configurations include: B512, B800, B1024, B1152, B1600, B2304, B3136, and B4096, where the nomenclature indicates the total number of MACs per DPU clock cycle.
- Software and IP core support for up to a maximum of four homogeneous DPU instances in a single AMD Xilinx® SoC.

The following list highlights key supported operators for the DPUCZDX8G :

- Supports both Convolution and transposed convolution
- Depthwise convolution and depthwise transposed convolution
- Max pooling
- Average pooling
- ReLU, ReLU6, Leaky ReLU, Hard Sigmoid, and Hard Swish
- Elementwise–sum and Elementwise–multiply
- Dilation
- Reorg
- Correlation 1D and 2D
- Argmax and Max along channel dimension
- Fully connected layer
- Softmax
- Concat, Batch Normalization

IP Facts

DPUCZDX8G IP Facts Table	
Core Specifics	
Supported Device Family	Zynq® UltraScale+™ MPSoC Family
Supported User Interfaces	Memory–mapped AXI interfaces
Resources	See DPU Configuration .
Provided with Core	
Design Files	Encrypted RTL
Example Design	Verilog
Constraints File	Xilinx Design Constraints (XDC)
Supported S/W Driver	Included in PetaLinux
Test Bench	Not Provided
Simulation Model	Not Provided
Tested Design Flows	

DPUCZDX8G IP Facts Table

Design Entry	Vivado® Design Suite and Vitis™ unified software platform
Simulation	N/A
Synthesis	Vivado® Synthesis

[Xilinx Support web page](#)

1. Linux OS and driver support information are available from DPUCZDX8G TRD or Vitis™ AI development kit.
2. For the supported tool versions, see the Vivado Design Suite User Guide: Release Notes, Installation, and Licensing ([UG973](#)).
3. The DPUCZDX8G is driven by instructions generated by the Vitis AI compiler. When the target neural network (NN), DPUCZDX8G hardware architecture, or AXI data width is changed, the related .xmodel file which contains DPUCZDX8G instructions must be regenerated with the updated arch.json file.
4. The DPU does not support hw_emu function. The reasons are as follows:
 - The RTL code of DPU is encrypted. Vitis™ does not analyze the source code.
 - The DPU is a co-processor and would require a highly complex test environment for adequate design verification.

Overview

Core Overview

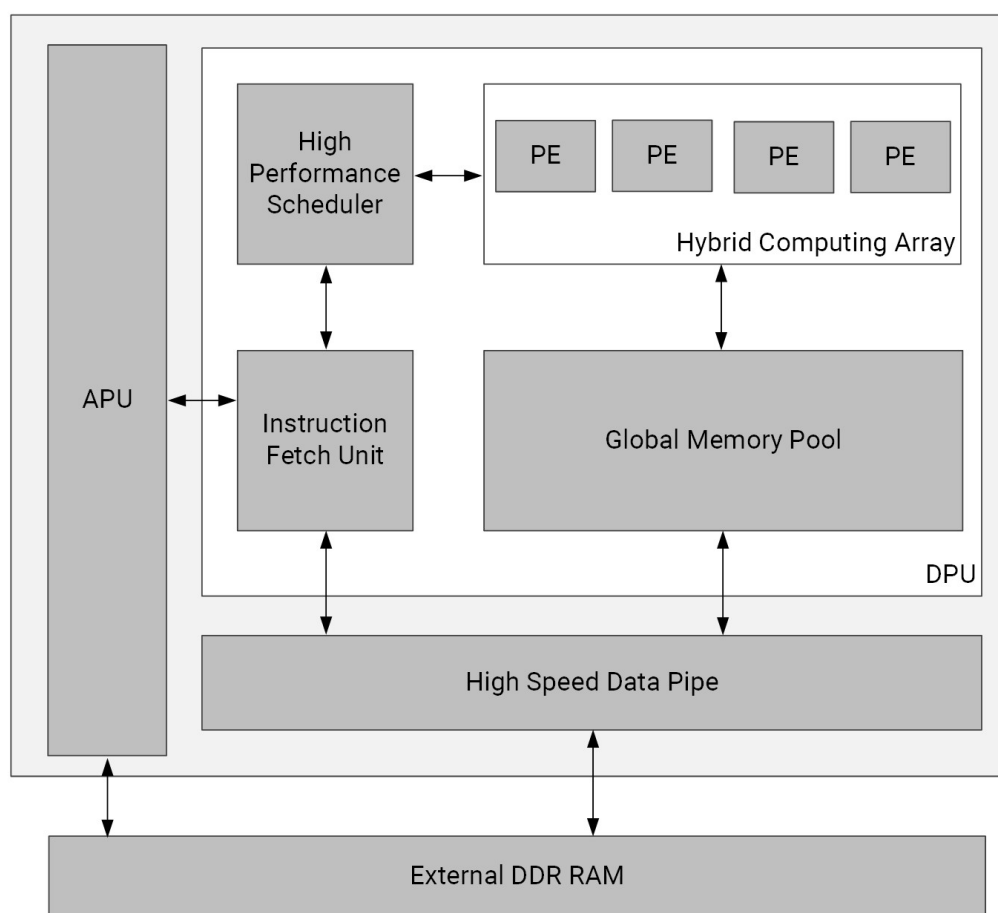
The Xilinx® DPUCZDX8G is a programmable engine optimized for convolutional neural networks. It is composed of a high performance scheduler module, a hybrid computing array module, an instruction fetch unit module, and a global memory pool module. The DPUCZDX8G is a microcoded engine which uses a specialized instruction set, which allows for the efficient implementation of many convolutional neural networks. Some examples of convolutional neural networks which have been deployed

include VGG, ResNet, GoogLeNet, YOLO, SSD, MobileNet, and FPN among others.

The DPUCZDX8G IP is implemented in the programmable logic (PL) of the selected Zynq® UltraScale+™ MPSoC device with direct connections to the processing system (PS). The DPUCZDX8G executes compiled microcode generated from a neural network graph, and requires accessible memory locations for input images as well as temporary and output data. A program running on the application processing unit (APU) is also required to service interrupts and coordinate data transfers.

The top-level block diagram of the DPUCZDX8G is shown in the following figure.

Figure: DPUCZDX8G Top-Level Block Diagram



where,

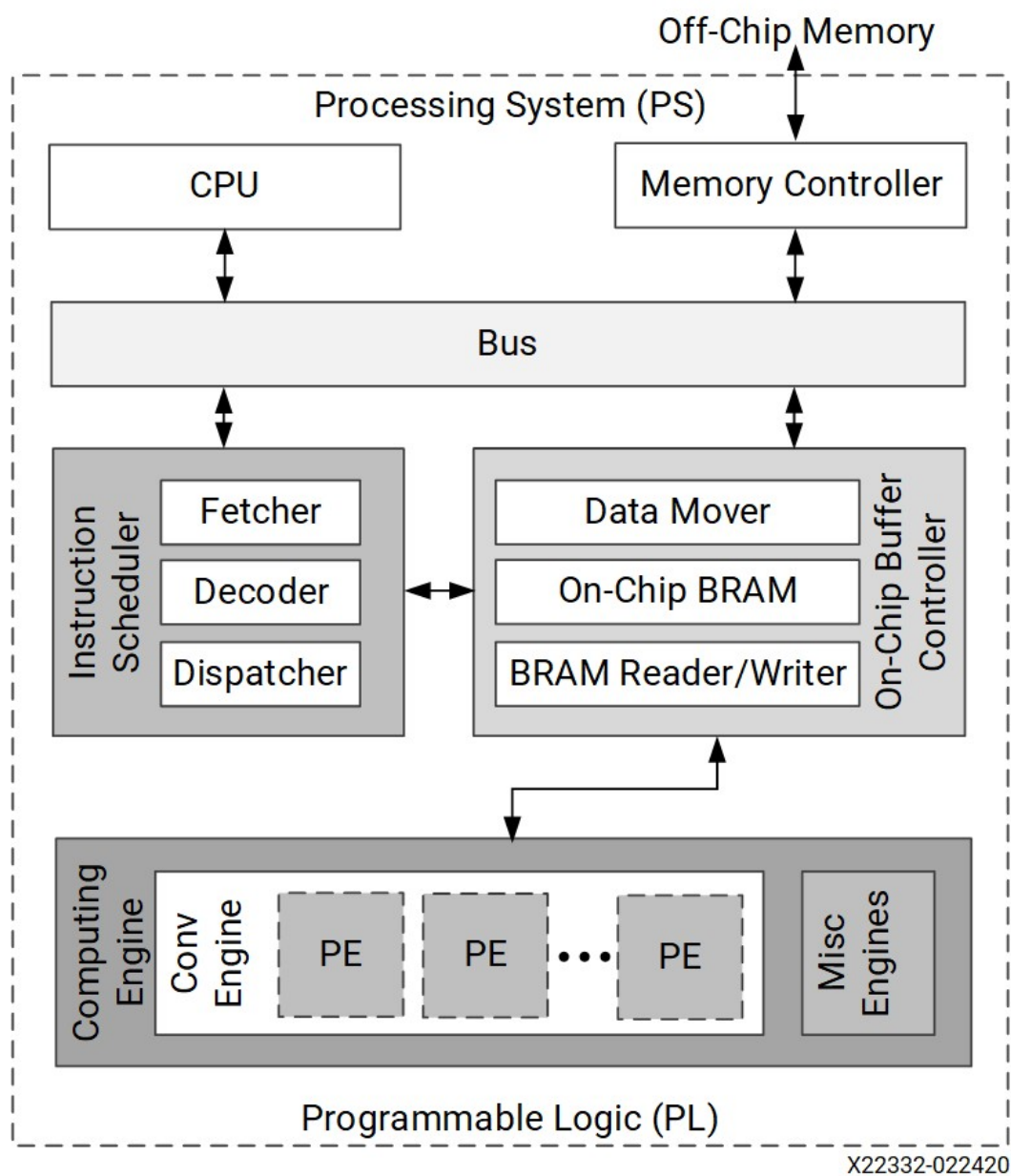
- APU – Application Processing Unit
- PE – Processing Engine
- DPU – Deep Learning Processing Unit

Hardware Architecture

The detailed hardware architecture of the DPUCZDX8G is shown in the following figure. After start-up, the DPUCZDX8G fetches instructions from off-chip memory to control the operation of the computing engine. The instructions are generated by the Vitis™ AI compiler, which performs substantial optimizations including layer fusion.

On-chip memory is used to buffer input activations, intermediate feature-maps, and output meta-data to achieve high throughput and efficiency. The data is reused as much as possible to reduce external memory bandwidth requirements. A deep pipelined design is used for the computing engine. The processing elements (PEs) take full advantage of the fine-grained building blocks such as multipliers, adders, and accumulators in Xilinx devices.

Figure: DPUCZDX8G Hardware Architecture



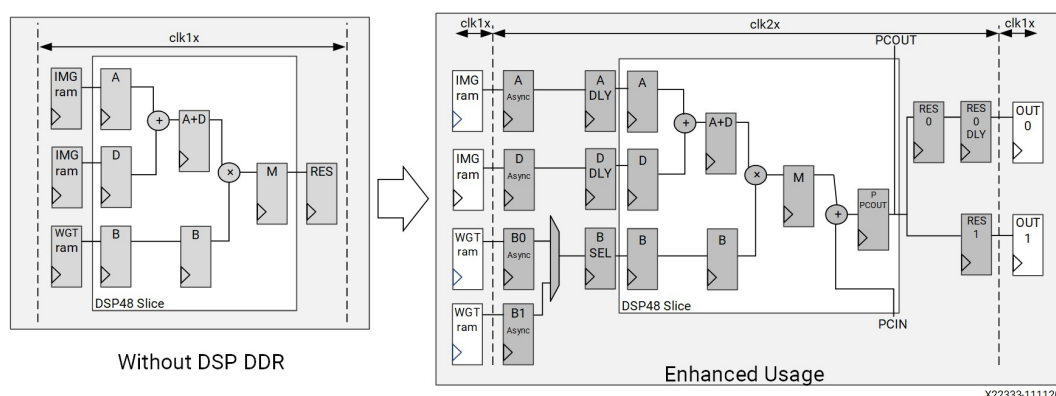
DPUCZDX8G with Enhanced Usage of DSP

A DSP Double Data Rate (DDR) technique may be used to improve the performance achieved with the DPU. In this configuration, two input clocks for the DPUCZDX8G are needed: A 1x clock for general logic and a 2x clock for DSP slices are employed. For more information, see [Clocking](#).

The difference between a DPUCZDX8G not using the DSP DDR technique and a DPUCZDX8G enhanced usage architecture is shown in the following image.

Note: All DPUCZDX8G architectures referred to in this document refer to DPUCZDX8G enhanced usage, unless otherwise specified.

Figure: DPUCZDX8G with (Enhanced Usage) and without DSP DDR



Development Tools

Two flows are supported for integrating the DPUCZDX8G into your project: the Vivado flow and the Vitis™ flow.

The Xilinx Vivado® Design Suite is required to integrate the DPUCZDX8G into your projects for the Vivado flow. Vivado Design Suite 2022.2 or later version is recommended. Contact your local sales representative if the project requires an older version of Vivado.

The Vitis unified software platform 2022.2 or later is required to integrate the DPUCZDX8G for the Vitis flow.

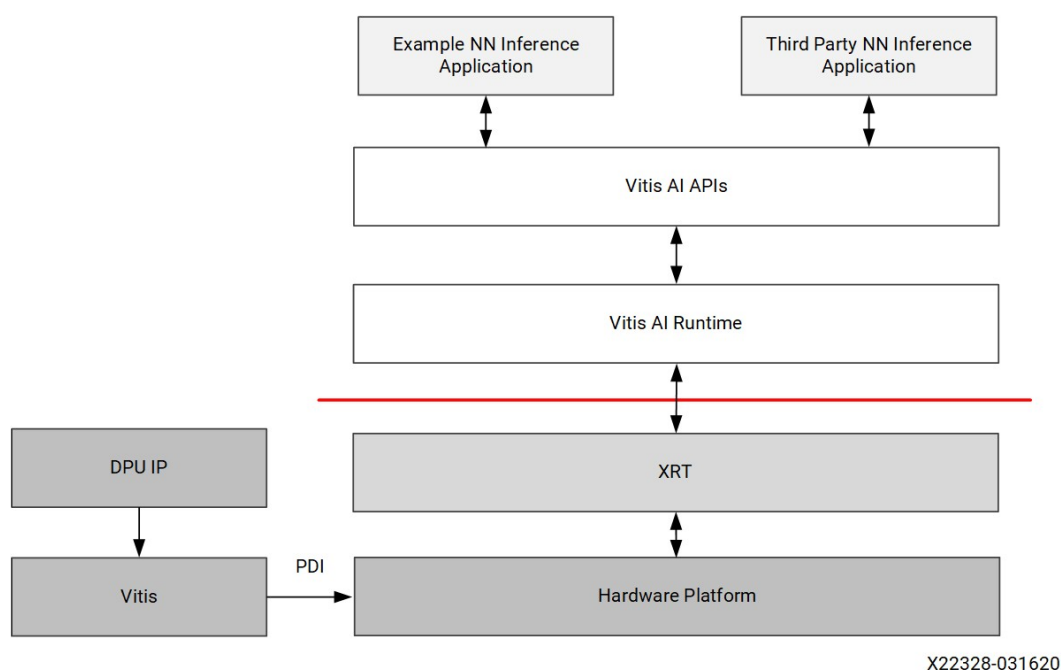
DPUCZDX8G Development Flow

The DPUCZDX8G requires a device driver which is included in the Xilinx Vitis™ AI development kit.

Free developer resources can be obtained from the Xilinx website: <https://github.com/Xilinx/Vitis-AI>.

The Vitis AI User Guide (UG1414) describes how to use the DPUCZDX8G with the Vitis AI tools. The basic development flow is shown in the following figure. First, use Vivado or Vitis to generate the bitstream. Then, download the bitstream to the target board and install the related driver. For instructions on installing the related driver and dependent libraries, see the Vitis AI User Guide (UG1414).

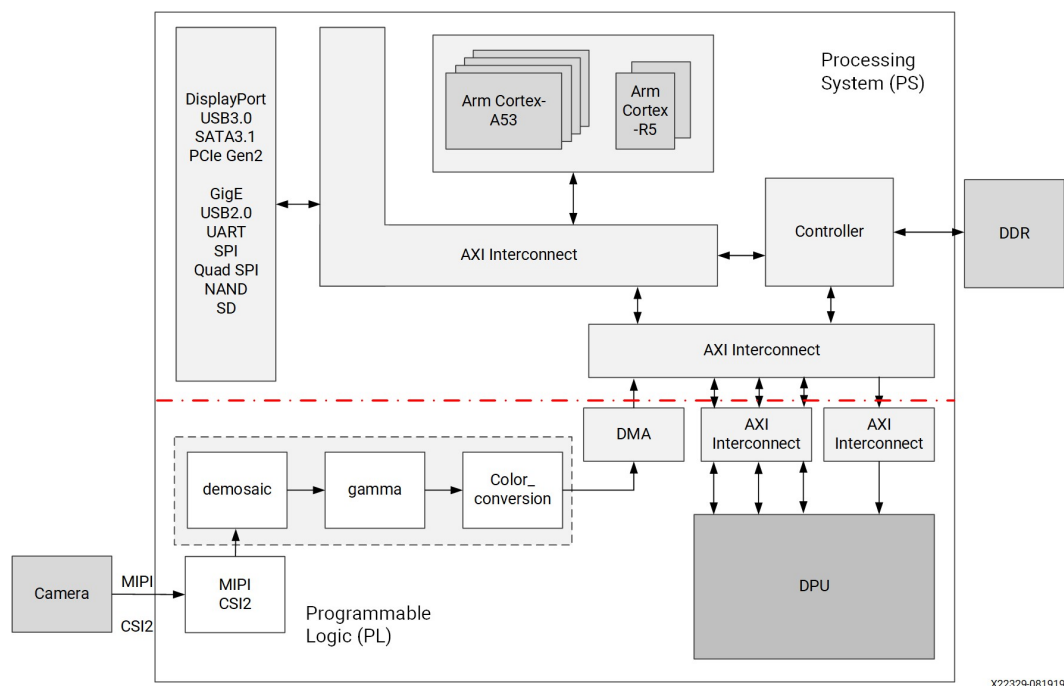
Figure: HW/SW Stack



MPSoC Integration Example

The following image illustrates an example system in which the video frames are captured by a Zynq® UltraScale+™ MPSoC device with a connected MIPI camera. The DPUCZDX8G is integrated into the system through an AXI interconnect to perform deep learning inference tasks such as image classification, object detection, and semantic segmentation.

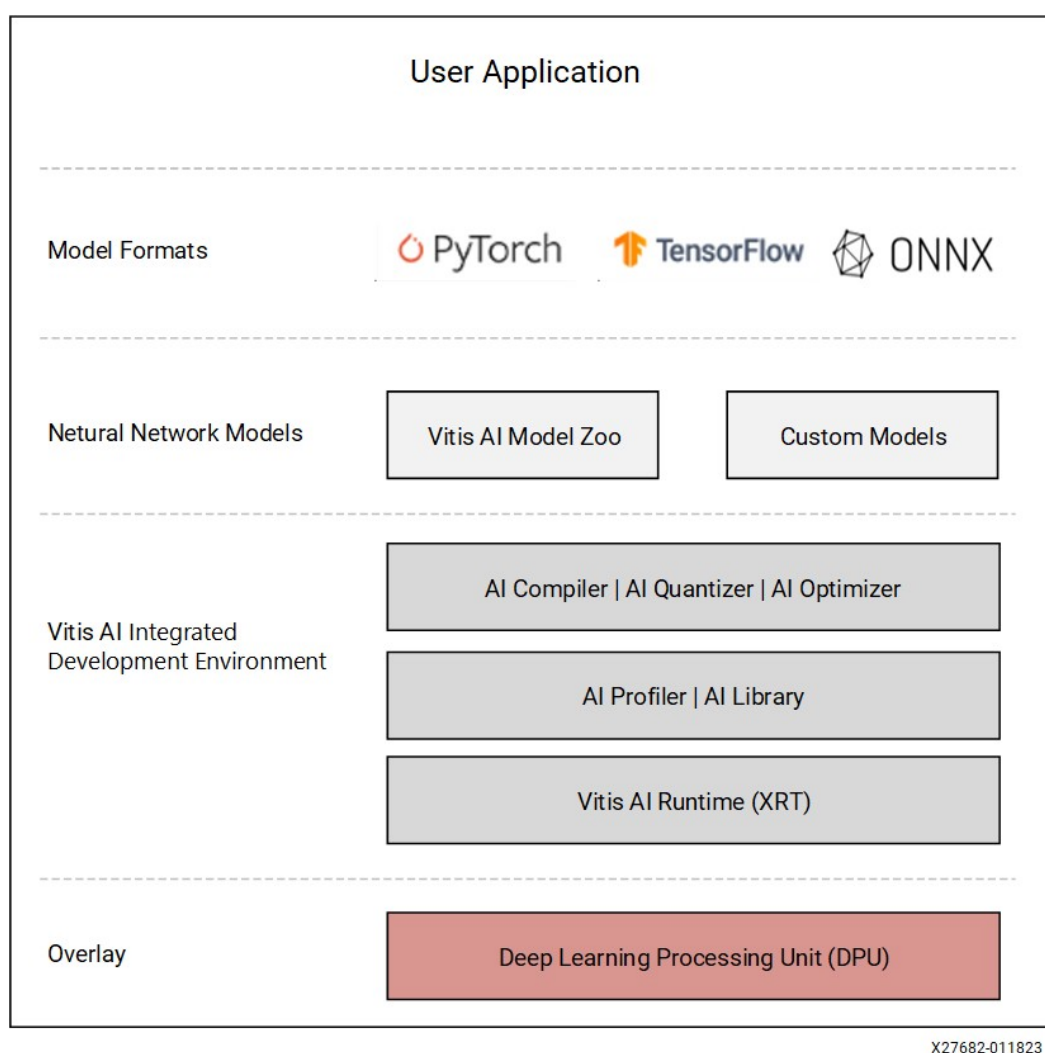
Figure: Example System with Integrated DPUCZDX8G



Vitis AI Development Kit

The Vitis™ AI development environment is used for AI inference on Xilinx® hardware platforms. It consists of optimized IP cores, tools, libraries, models, and example designs.

As shown in the following figure, the Vitis AI development kit consists of the AI Compiler, AI Quantizer, AI Optimizer, AI Profiler, AI Library, and Xilinx Runtime Library (XRT).

Figure: Vitis AI Stack

Further details of the Vitis AI development kit can be found in the Vitis AI User Guide ([UG1414](#)).

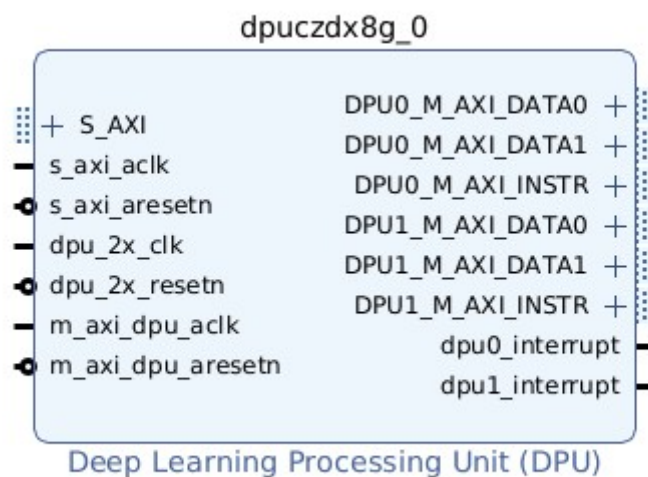
Note: The Vitis AI development kit is a free download, available from [here](#).

Product Specification

Port Descriptions

The DPUCZDX8G top-level interfaces are shown in the following figure.

Figure: Two DPU Kernel Ports



The DPUCZDX8G I/O signals are listed and described in the table below.

Table: DPUCZDX8G Signal Description

Signal Name	Interface Type	Width	I/O	Description
S_AXI	Memory mapped AXI slave interface	32	I/O	32-bit memory mapped AXI interface for registers.
s_axi_aclk	Clock	1	I	AXI clock input for S_AXI
s_axi_aresetn	Reset	1	I	Active-Low reset for S_AXI

Signal Name	Interface Type	Width	I/O	Description
dpu_2x_clk	Clock	1	I	Input clock used for DSP blocks in the DPUCZDX8G . The frequency is twice that of m_axi_dpu_aclk.
dpu_2x_resetn	Reset	1	I	Active-Low reset for DSP blocks
m_axi_dpu_aclk	Clock	1	I	Input clock used for DPUCZDX8G general logic.
m_axi_dpu_aresetn	Reset	1	I	Active-Low reset for DPUCZDX8G general logic
DPUx_M_AXI_INSTR	Memory mapped AXI master interface	32	I/O	32-bit memory mapped AXI interface for DPUCZDX8G instruction fetch.

Signal Name	Interface Type	Width	I/O	Description
DPUx_M_AXI_DATA0	Memory mapped AXI master interface	128	I/O	128-bit for Zynq UltraScale+ MPSoC series.
DPUx_M_AXI_DATA1	Memory mapped AXI master interface	128	I/O	128-bit for Zynq UltraScale+ MPSoC series.
dpux_interrupt	Interrupt	1	O	Active-High interrupt output from DPUCZDX8G.
SFM_M_AXI (optional)	Memory mapped AXI master interface	128	I/O	Memory-mapped AXI interface for softmax data.
sfm_interrupt (optional)	Interrupt	1	O	Active-High interrupt output from softmax module.

Signal Name	Interface Type	Width	I/O	Description
dpu_2x_clk_ ce (optional)	Clock enable	1	O	Active-High clock-enable which is used to gate the 2x_clk input when clock gating is enabled.

1. The softmax interface is present only when the softmax option for the DPUCZDX8G is enabled.

Register Space

The DPUCZDX8G IP implements registers in programmable logic. The following tables show the DPUCZDX8G IP registers. These registers are accessible from the APU through the S_AXI interface.

reg_dpu_reset

The reg_dpu_reset register controls the resets of all DPUCZDX8G cores integrated in the DPUCZDX8G IP. The lower four bits of this register control the reset of up to four DPUCZDX8G cores. All the reset signals are active-High. The details of reg_dpu_reset are shown in the following table.

Table: reg_dpu_reset

Register	Address Offset	Width	Type	Description
----------	----------------	-------	------	-------------

Register	Address Offset	Width	Type	Description
reg_dpu_reset	0x004	32	R/W	[n] – DPUCZDX8G core n reset

reg_dpu_isr

The reg_dpu_isr register represents the interrupt status of all cores in the DPUCZDX8G IP. The lower four bits of this register shows the interrupt status of up to four DPUCZDX8G cores. The details of reg_dpu_irq are shown in the following table.

Table: reg_dpu_isr

Register	Address Offset	Width	Type	Description
reg_dpu_isr	0x608	32	R	[n] – DPUCZDX8G core n interrupt status

reg_dpu_start

The reg_dpu_start register is the start signal for a DPUCZDX8G core. There is one start register for each DPUCZDX8G core. The details of reg_dpu_start are shown in the following table.

Table: reg_dpu_start

Register	Address Offset	Width	Type	Description
----------	----------------	-------	------	-------------

Register	Address Offset	Width	Type	Description
reg_dpu0_start	0x220	32	R/W	DPUCZDX8G core0 start signal.
reg_dpu1_start	0x320	32	R/W	DPUCZDX8G core1 start signal.
reg_dpu2_start	0x420	32	R/W	DPUCZDX8G core2 start signal.
reg_dpu3_start	0x520	32	R/W	DPUCZDX8G core3 start signal.

reg_dpu_instr_addr

The reg_dpu_instr_addr register is used to indicate the instruction address of a DPUCZDX8G core. Each DPUCZDX8G core has a reg_dpu_instr_addr register. Only the lower 28-bits are valid. In the DPUCZDX8G processor, the real instruction-fetch address is a 40-bit signal which consists of the lower 28 bits of reg_dpu_instr_addr followed by 12 zero bits. The available instruction address for DPU ranges from 0x1000 to 0xFFFF_FFFF_FFFF_F000. The details of reg_dpu_instr_addr are shown in the following table.

Table: reg_dpu_instr_addr

Register	Address Offset	Width	Type	Description
----------	----------------	-------	------	-------------

Register	Address Offset	Width	Type	Description
reg_dpu0_instr_addr	0x20C	32	R/W	Start address in external memory for DPUCZDX8G core0 instructions. The lower 28-bit is valid.
reg_dpu1_instr_addr	0x30C	32	R/W	Start address in external memory for DPUCZDX8G core1 instructions. The lower 28-bit is valid.
reg_dpu2_instr_addr	0x40C	32	R/W	Start address in external memory for DPUCZDX8G core2 instructions. The lower 28-bit is valid.

Register	Address Offset	Width	Type	Description
reg_dpu3_in_str_addr	0x50C	32	R/W	Start address in external memory for DPUCZDX8G core3 instructions. The lower 28-bit is valid.

reg_dpu_base_addr

The reg_dpu_base_addr register is used to indicate the address of input image and parameters for each DPUCZDX8G in external memory. The width of a DPUCZDX8G base address is 40 bits so it can support an address space up to 1 TB. All registers are 32 bits wide, so two registers are required to represent a 40-bit wide base address. reg_dpu0_base_addr0_l represents the lower 32 bits of base_address0 in DPUCZDX8G core0 and reg_dpu0_base_addr0_h represents the upper eight bits of base_address0 in DPUCZDX8G core0.

There are eight groups of DPUCZDX8G base addresses for each DPUCZDX8G core and thus 32 groups of DPUCZDX8G base addresses for up to four DPUCZDX8G cores. The details of reg_dpu_base_addr are shown in the following table.

Table: reg_dpu_base_addr

Register	Address Offset	Width	Type	Description
----------	----------------	-------	------	-------------

Register	Address Offset	Width	Type	Description
reg_dpu0_base_addr0_l	0x224	32	R/W	The lower 32 bits of base_address0 of DPUCZDX8G core0.
reg_dpu0_base_addr0_h	0x228	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_address0 of DPUCZDX8G core0.
reg_dpu0_base_addr1_l	0x22C	32	R/W	The lower 32 bits of base_address1 of DPUCZDX8G core0.
reg_dpu0_base_addr1_h	0x230	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_address1 of DPUCZDX8G core0.

Register	Address Offset	Width	Type	Description
reg_dpu0_base_addr2_l	0x234	32	R/W	The lower 32 bits of base_addresses2 of DPUCZDX8G core0.
reg_dpu0_base_addr2_h	0x238	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses2 of DPUCZDX8G core0.
reg_dpu0_base_addr3_l	0x23C	32	R/W	The lower 32 bits of base_addresses3 of DPUCZDX8G core0.
reg_dpu0_base_addr3_h	0x240	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses3 of DPUCZDX8G core0.

Register	Address Offset	Width	Type	Description
reg_dpu0_base_addr4_l	0x244	32	R/W	The lower 32 bits of base_addresses4 of DPUCZDX8G core0.
reg_dpu0_base_addr4_h	0x248	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses4 of DPUCZDX8G core0.
reg_dpu0_base_addr5_l	0x24C	32	R/W	The lower 32 bits of base_addresses5 of DPUCZDX8G core0.
reg_dpu0_base_addr5_h	0x250	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses5 of DPUCZDX8G core0.

Register	Address Offset	Width	Type	Description
reg_dpu0_base_addr6_l	0x254	32	R/W	The lower 32 bits of base_addresses6 of DPUCZDX8G core0.
reg_dpu0_base_addr6_h	0x258	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses6 of DPUCZDX8G core0.
reg_dpu0_base_addr7_l	0x25C	32	R/W	The lower 32 bits of base_addresses7 of DPUCZDX8G core0.
reg_dpu0_base_addr7_h	0x260	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses7 of DPUCZDX8G core0.

Register	Address Offset	Width	Type	Description
reg_dpu1_base_addr0_l	0x324	32	R/W	The lower 32 bits of base_addresses0 of DPUCZDX8G core1.
reg_dpu1_base_addr0_h	0x328	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses0 of DPUCZDX8G core1.
reg_dpu1_base_addr1_l	0x32C	32	R/W	The lower 32 bits of base_addresses1 of DPUCZDX8G core1.
reg_dpu1_base_addr1_h	0x330	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses1 of DPUCZDX8G core1.

Register	Address Offset	Width	Type	Description
reg_dpu1_base_addr2_l	0x334	32	R/W	The lower 32 bits of base_addresses2 of DPUCZDX8G core1.
reg_dpu1_base_addr2_h	0x338	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses2 of DPUCZDX8G core1.
reg_dpu1_base_addr3_l	0x33C	32	R/W	The lower 32 bits of base_addresses3 of DPUCZDX8G core1.
reg_dpu1_base_addr3_h	0x340	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses3 of DPUCZDX8G core1.

Register	Address Offset	Width	Type	Description
reg_dpu1_base_addr4_l	0x344	32	R/W	The lower 32 bits of base_addresses4 of DPUCZDX8G core1.
reg_dpu1_base_addr4_h	0x348	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses4 of DPUCZDX8G core1.
reg_dpu1_base_addr5_l	0x34C	32	R/W	The lower 32 bits of base_addresses5 of DPUCZDX8G core1.
reg_dpu1_base_addr5_h	0x350	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses5 of DPUCZDX8G core1.

Register	Address Offset	Width	Type	Description
reg_dpu1_base_addr6_l	0x354	32	R/W	The lower 32 bits of base_addresses6 of DPUCZDX8G core1.
reg_dpu1_base_addr6_h	0x358	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses6 of DPUCZDX8G core1.
reg_dpu1_base_addr7_l	0x35C	32	R/W	The lower 32 bits of base_addresses7 of DPUCZDX8G core1.
reg_dpu1_base_addr7_h	0x360	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses7 of DPUCZDX8G core1.

Register	Address Offset	Width	Type	Description
reg_dpu2_base_addr1_l	0x42C	32	R/W	The lower 32 bits of base_addresses1 of DPUCZDX8G core2.
reg_dpu2_base_addr1_h	0x430	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses1 of DPUCZDX8G core2.
reg_dpu2_base_addr2_l	0x434	32	R/W	The lower 32 bits of base_addresses2 of DPUCZDX8G core2.
reg_dpu2_base_addr2_h	0x438	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses2 of DPUCZDX8G core2.

Register	Address Offset	Width	Type	Description
reg_dpu2_base_addr3_l	0x43C	32	R/W	The lower 32 bits of base_addresses3 of DPUCZDX8G core2.
reg_dpu2_base_addr3_h	0x440	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses3 of DPUCZDX8G core2.
reg_dpu2_base_addr4_l	0x444	32	R/W	The lower 32 bits of base_addresses4 of DPUCZDX8G core2.
reg_dpu2_base_addr4_h	0x448	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses4 of DPUCZDX8G core2.

Register	Address Offset	Width	Type	Description
reg_dpu2_base_addr5_l	0x44C	32	R/W	The lower 32 bits of base_addresses5 of DPUCZDX8G core2.
reg_dpu2_base_addr5_h	0x450	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses5 of DPUCZDX8G core2.
reg_dpu2_base_addr6_l	0x454	32	R/W	The lower 32 bits of base_addresses6 of DPUCZDX8G core2.
reg_dpu2_base_addr6_h	0x458	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses6 of DPUCZDX8G core2.

Register	Address Offset	Width	Type	Description
reg_dpu2_base_addr7_l	0x45C	32	R/W	The lower 32 bits of base_addresses7 of DPUCZDX8G core2.
reg_dpu2_base_addr7_h	0x460	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses7 of DPUCZDX8G core2.
reg_dpu3_base_addr0_l	0x524	32	R/W	The lower 32 bits of base_addresses0 of DPUCZDX8G core3.
reg_dpu3_base_addr0_h	0x528	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses0 of DPUCZDX8G core3.

Register	Address Offset	Width	Type	Description
reg_dpu3_base_addr1_l	0x52C	32	R/W	The lower 32 bits of base_addresses1 of DPUCZDX8G core3.
reg_dpu3_base_addr1_h	0x530	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses1 of DPUCZDX8G core3.
reg_dpu3_base_addr2_l	0x534	32	R/W	The lower 32 bits of base_addresses2 of DPUCZDX8G core3.
reg_dpu3_base_addr2_h	0x538	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses2 of DPUCZDX8G core3.

Register	Address Offset	Width	Type	Description
reg_dpu3_base_addr3_l	0x53C	32	R/W	The lower 32 bits of base_addresses3 of DPUCZDX8G core3.
reg_dpu3_base_addr3_h	0x540	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses3 of DPUCZDX8G core3.
reg_dpu3_base_addr4_l	0x544	32	R/W	The lower 32 bits of base_addresses4 of DPUCZDX8G core3.
reg_dpu3_base_addr4_h	0x548	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses4 of DPUCZDX8G core3.

Register	Address Offset	Width	Type	Description
reg_dpu3_base_addr5_l	0x54C	32	R/W	The lower 32 bits of base_addresses5 of DPUCZDX8G core3.
reg_dpu3_base_addr5_h	0x550	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses5 of DPUCZDX8G core3.
reg_dpu3_base_addr6_l	0x554	32	R/W	The lower 32 bits of base_addresses6 of DPUCZDX8G core3
reg_dpu3_base_addr6_h	0x558	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses6 of DPUCZDX8G core3.

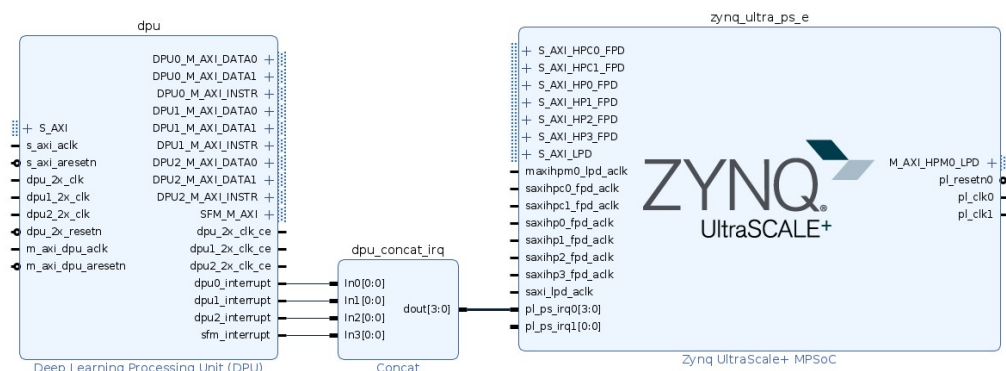
Register	Address Offset	Width	Type	Description
reg_dpu3_base_addr7_l	0x55C	32	R/W	The lower 32 bits of base_addresses7 of DPUCZDX8G core3.
reg_dpu3_base_addr7_h	0x560	32	R/W	The lower 8 bits in the register represent the upper 8 bits of base_addresses7 of DPUCZDX8G core3.

Interrupts

A high state on reg_dpu0_start signals the start of a DPUCZDX8G task for DPUCZDX8G core0. At the end of the task, the DPUCZDX8G generates an interrupt to signal the completion of a task, and bit0 in reg_dpu_isr is set to 1. The position of the active bit in the reg_dpu_isr depends on the number of DPUCZDX8G cores. For example, when DPUCZDX8G core1 finishes a task while DPUCZDX8G core0 is still processing, reg_dpu_isr would contain 2'b10.

The DPU IP may be configured to instantiate 1–4 cores (see [DPU Configuration](#)). The IP wrapper for the configured DPU will have up to four dpux_interrupt signals, corresponding to the number of cores. These signals must be routed through a concat block, and then be connected to the PS as illustrated in the following image.

Figure: Interrupt Concatenation for Multiple DPU Cores



Note:

1. irq7~irq0 corresponds to pl_ps_irq0[7:0].
2. irq15~irq8 corresponds to pl_ps_irq1[7:0].

DPU Configuration

Introduction

The DPUCZDX8G IP provides some user-configurable parameters to optimize resource usage and customize different features. Different configurations can be selected for DSP slices, LUT, block RAM, and UltraRAM usage based on the amount of available programmable logic resources. There are also options for additional functions, such as channel augmentation, average pooling, depthwise convolution, and softmax. Furthermore, there is an option to determine the number of DPUCZDX8G cores that will be instantiated in a single DPUCZDX8G IP.

The deep neural network features and the associated parameters supported by the DPUCZDX8G are shown in the following table.

A configuration file named arch.json is generated during the Vivado or Vitis flow. The arch.json file is used by the Vitis AI Compiler for model compilation. If the content of arch.json is changed. Please re-compile the

model. For more information of Vitis AI Compiler, see refer to the Vitis AI User Guide ([UG1414](#)).

In the Vivado flow, the arch.json file is located at \$TRD_HOME/prj/Vivado/srcs/top/ip/top_dpu_0/arch.json.

In the Vitis flow, the arch.json file is located at \$TRD_HOME/prj/Vitis/binary_container_1/link/vivado/vpl/prj/prj.gen/sources_1/bd/zcu102_base/ip/zcu102_base_DPUCZDX8G_1_0/arch.json.

 **Note:** \$TRD_home = Vitis-AI/dsa/DPU-TRD/

Table: DPUCZDX8G Operation and Parameter Support

Features		Description
Convolution	Kernel Sizes	w, h: [1, 16]
	Strides	w, h: [1, 8]
	Padding	w: [0, kernel_w-1] h: [0, kernel_h-1]
	Input Size	Arbitrary
	Input Channel	kernel_w * kernel_h * ceil(input_channel / channel_parallel) <= bank_depth
	Output Channel	1~256 * channel_parallel
	Activation	ReLU, ReLU6, LeakyReLU, Hard Sigmoid and Hard Swish

Features	Description	
	Dilation	dilation * input_channel $\leq$ 256 * channel_parallel && stride_w == 1 && stride_h == 1
Depthwise Convolution	Kernel Sizes	w, h: [1, 256]
	Strides	w, h: [1, 256]
	Padding	w: [0, min(kernel_w-1,15)] h: [0, min(kernel_h-1,15)]
	Input Size	Arbitrary
	Input Channel	kernel_w * kernel_h * (ceil(input_channel / channel_parallel)) <= bank_depth
	Output Channel	1~256 * channel_parallel
	Activation	ReLU, ReLU6, LeakyReLU, Hard Sigmoid and Hard Swish
	Dilation	dilation * input_channel $\leq$ 256 * channel_parallel && stride_w == 1 && stride_h == 1
Transposed Convolution	Kernel Sizes	kernel_w/stride_w: [1, 16]

Features		Description
	Strides	kernel_h/stride_h: [1, 16]
	Padding	w: [0, kernel_w-1] h: [0, kernel_h-1]
	Input Size	Arbitrary
	Input Channel	kernel_w * kernel_h * (ceil(input_channel / channel_parallel)) <= bank_depth
	Output Channel	1~256 * channel_parallel
	Activation	ReLU, ReLU6 and LeakyReLU
Depthwise Transposed Convolution	Kernel Sizes	kernel_w/stride_w: [1, 16]
	Strides	kernel_h/stride_h: [1, 16]
	Padding	w: [0, kernel_w-1] h: [0, kernel_h-1]
	Input Size	Arbitrary
	Input Channel	kernel_w * kernel_h * (ceil(input_channel / channel_parallel)) <= bank_depth
	Output Channel	1~256 * channel_parallel

Features	Description	
	Activation	ReLU, ReLU6, LeakyRelu, Hard Sigmoid and Hard Swish
Max Pooling	Kernel Sizes	w, h: [1, 256]
	Strides	w, h: [1, 256]
	Padding	w: [0, min(kernel_w-1, 15)] h: [0, min(kernel_h-1, 15)]
Average Pooling	Kernel Sizes	w, h: [1, 256]
	Strides	w, h: [1, 256]
	Padding	w: [0, min(kernel_w-1, 15)] h: [0, min(kernel_h-1, 15)]
Elementwise-Sum	Input channel	1~256 * channel_parallel
	Input size	Arbitrary
	Feature Map Number	1~4
Elementwise-Multiply	Input channel	1~256 * channel_parallel
	Input size	Arbitrary
	Feature Map Number	2
Concat	Output channel	1~256 * channel_parallel

Features		Description
Reorg	Strides	$\text{stride} * \text{stride} * \text{input_channel} \leq 256 * \text{channel_parallel}$
Fully Connected	Input_channel	$\text{Input_channel} \leq 2048 * \text{channel_parallel}$
	Output_channel	Arbitrary

1.

The parameter channel_parallel is determined by the DPUCZDX8G configuration. For example, channel_parallel for the B1152 is 12, and channel_parallel for B4096 is 16 (see Parallelism for Different Convolution Architectures table in Configuration Options section).
2.

In some neural networks, the FC layer is connected with a Flatten layer. The Vitis AI compiler will automatically combine Flatten+FC to a global CONV2D layer, and the CONV2D kernel size is equal to the input feature map size of Flatten layer. For this case, the input feature map size cannot exceed this automatically configured CONV2D kernel size, otherwise an error will be generated during compilation. If there is no flatten layer, the FC layer will be treated as a normal conv layer.
This limitation is present only in cases where the Flatten and FC layers are fused by the compiler.
3.

The bank_depth refers to the on-chip weight buffer depth. In all DPU architectures, the bank_depth of the feature map and weights is 2048.
4.

If Batch Normalization operators can be transformed by nature of equivalency to depthwise-conv-2d, the operator will be transformed for execution on the DPU. Otherwise, batch_norm will be executed by CPU.

Related Information

[Configuration Options](#)

Configuration Options

The DPUCZDX8G can be configured with some predefined options, which includes the number of DPUCZDX8G cores, the convolution architecture, DSP cascade, DSP usage, and UltraRAM usage. These options allow you to set the DSP slice, LUT, block RAM, and UltraRAM usage. The following figure shows the configuration page of the DPUCZDX8G.

Figure: DPUCZDX8G Configuration – Arch Tab

The screenshot displays the configuration interface for the DPUCZDX8G_0 component. On the left, a block diagram shows the component with various input and output ports. On the right, the configuration options are organized into sections:

- Arch** (selected tab):
 - Number of DPU Cores: 1
 - Arch of DPU: B4096
 - RAM Usage: Low
 - Channel Augmentation: Enabled
 - Save Argmax: Enabled
- Conv**:
 - Conv ReLU Type: ReLU + LeakyReLU + ReLU6
- Alu**:
 - Alu Parallel: 4
 - Alu Parallel Actual: 4
 - ElementWise Multiply: Enabled
 - AveragePool: Enabled
 - Alu ReLU Type: ReLU + ReLU6
- Softmax**:
 - Number of SFM cores: 0

The following sections describe the configuration options.

Number of DPU Cores

A maximum of four cores can be selected in one DPUCZDX8G IP. Multiple DPUCZDX8G cores can be used to achieve higher performance, at the expense of programmable logic resources.

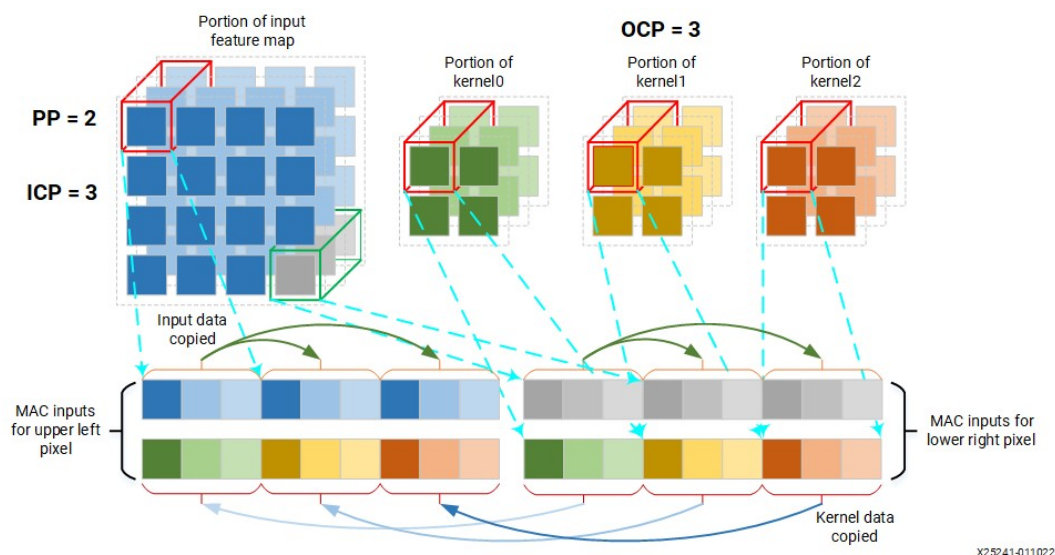
Contact your local Xilinx sales representative if you require more than four cores.

Architecture of the DPUCZDX8G

The DPUCZDX8G IP can be configured with various convolution architectures which are related to the parallelism of the convolution unit. The architectures for the DPUCZDX8G IP include B512, B800, B1024, B1152, B1600, B2304, B3136, and B4096.

There are three dimensions of parallelism in the DPUCZDX8G convolution architecture: pixel parallelism, input channel parallelism, and output channel parallelism. The input channel parallelism is always equal to the output channel parallelism (this is equivalent to `channel_parallel` in Table 8).

Figure: Visualizing the three dimensions of parallelism



In Figure 10, input channel parallelism (ICP) = 3; output channel parallelism (OCP) = 3; and pixel parallelism (PP) = 2. OCP is equivalent to the number of kernels used during a convolution computation. The pixels used in the figure are arbitrary to maintain clarity.

Note: The elements used in the computation use 1 pixel from each channel (the red cuboids in the figure). With ICP = OCP = 3 and PP = 2, the number of convolution MACs per cycle is $3 * 3 * 2 = 18$.

The different architectures require different programmable logic resources. The larger architectures can achieve higher performance with more

resources. The parallelism for the different architectures is listed in the following table.

Table: Parallelism for Different Convolution Architectures

DPUCZDX8G Architecture	Pixel Parallelism (PP)	Input Channel Parallelism (ICP)	Output Channel Parallelism (OCP)	Peak Ops (operations /per cycle)
B512	4	8	8	512
B800	4	10	10	800
B1024	8	8	8	1024
B1152	4	12	12	1152
B1600	8	10	10	1600
B2304	8	12	12	2304
B3136	8	14	14	3136
B4096	8	16	16	4096
1. In each clock cycle, the convolution array performs a multiplication and an accumulation, which are counted as two operations. Thus, the peak number of operations per cycle is equal to $PP \times ICP \times OCP \times 2$.				

Resource Utilization

The resource utilization of a sample DPUCZDX8G single core project is as follows. The data is based on the ZCU102 platform with low RAM usage, channel augmentation, alu parallel = $PP/2$, conv: leaky ReLU + ReLU6, alu: ReLU6 features, high DSP usage, and Argmax enabled.

Table: Resources of Different DPUCZDX8G Architectures

DPUCZDX8G Architecture	LUT	Register	Block RAM	DSP
B512	26922	34543	72	118
B800	29721	41147	90	166
B1024	34074	48057	104	230
B1152	32169	47374	121	222
B1600	38418	58831	126	326
B2304	42127	68829	165	438
B3136	46714	79710	208	566
B4096	52161	98249	255	710

Another example of a DPUCZDX8G single core project is based on the ZCU104 platform. In this project, the image and weights buffer utilize UltraRAM. The project is configured with low RAM usage, channel augmentation, alu parallel = PP/2, conv: leaky ReLU + ReLU6, alu: ReLU6 features, and high DSP usage. The resource utilization of this project is as follows.

Table: Resources of DPUCZDX8G using UltraRAM

DPUCZDX8G Architecture	LUT	Register	Block RAM	UltraRAM	DSP
B512	26767	34538	0	18	118
B800	29578	41153	0	40	166
B1024	33752	48093	0	26	230

DPUCZDX8G Architect ure	LUT	Register	Block RAM	UltraRAM	DSP
B1152	31901	47797	0	44	222
B1600	38197	58859	0	56	326
B2304	41736	69604	0	60	438
B3136	46260	80079	0	64	566
B4096	51843	98567	0	68	710

RAM Usage

The weights, bias, and intermediate feature maps are buffered in the on-chip memory. The on-chip memory consists of RAM which can be instantiated as block RAM and UltraRAM. The RAM Usage option determines the total amount of on-chip memory used in different DPUCZDX8G architectures, and the setting is for all the DPUCZDX8G cores in the DPUCZDX8G IP. High RAM Usage means that the on-chip memory block will be larger, allowing the DPUCZDX8G more flexibility in handling the intermediate data. High RAM Usage implies higher performance in each DPUCZDX8G core. The number of BRAM36K blocks used in different architectures for low and high RAM Usage is illustrated in the following table.

 **Note:** The DPUCZDX8G instruction set for different options of RAM Usage is different. When the RAM Usage option is modified, the DPUCZDX8G instructions file should be regenerated by recompiling the neural network. The following results are based on a DPUCZDX8G with depthwise convolution.

Table: Number of BRAM36K Blocks in Different Architectures for Each DPUCZDX8G Core

DPUCZDX8G Architecture	Low RAM Usage	High RAM Usage
B512	72	88
B800	90	108
B1024	104	136
B1152	121	143
B1600	126	162
B2304	165	209
B3136	208	260
B4096	255	315

Channel Augmentation

Channel augmentation is an optional feature for improving the efficiency of the DPUCZDX8G when the number of input channels is much lower than the available channel parallelism. For example, the input channel of the first layer in most CNNs is three, which does not fully use all the available hardware channels. If the number of input channels is larger than the channel parallelism, channel augmentation can still be utilized.

Thus, channel augmentation can improve the total efficiency for most CNNs, but it will cost extra logic resources. The following table illustrates the extra LUT resources used with channel augmentation and the statistics are for reference.

Table: Extra LUTs of DPUCZDX8G with Channel Augmentation

DPUCZDX8G Architecture	Extra LUTs with Channel Augmentation
------------------------	--------------------------------------

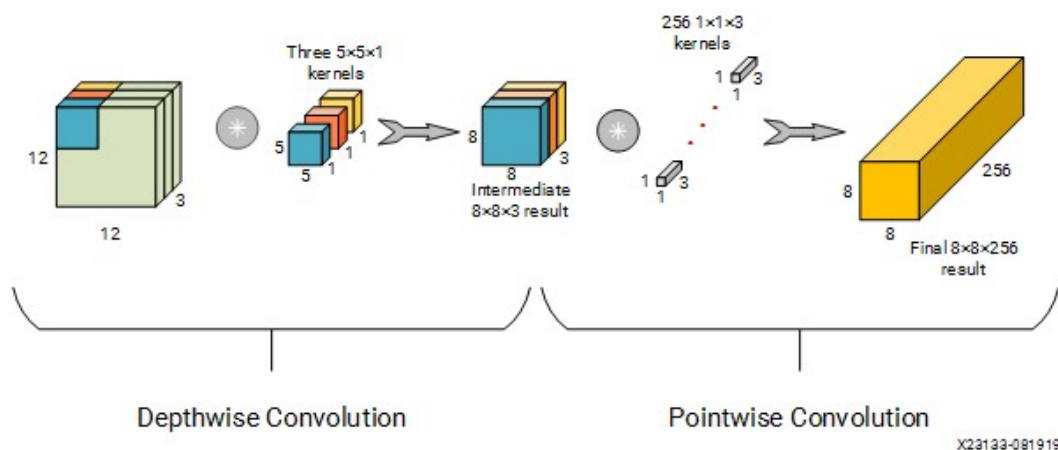
DPUCZDX8G Architecture	Extra LUTs with Channel Augmentation
B512	3121
B800	2624
B1024	3133
B1152	1744
B1600	2476
B2304	1710
B3136	1946
B4096	1701

DepthwiseConv (ALU)

In conventional convolution, each input channel needs to perform the operation with one specific kernel, and then the result is obtained by combining the results of all channels together.

In depthwise separable convolution, the operation is performed in two steps: depthwise convolution and pointwise convolution. Depthwise convolution is performed for each feature map separately as shown on the left side of the following figure. The next step is to perform pointwise convolution, which is the same as conventional convolution with kernel size 1x1. The parallelism of depthwise convolution is half that of the pixel parallelism.

In DPUCZDX8G, the depthwise conv is performed by the ALU engine, along with the pooling. The ALU parallel ranges from 1 to PP, and is recommended to be set as PP/2.

Figure: Depthwise Convolution and Pointwise Convolution**Table: Resources of DPUCZDX8G B4096 with Different ALU Parallel**

ALU Parallel	LUTs	FF	Block RAMs	DSPs
1	44212	88250	255	662
2	46599	92380	255	678
4 (recommended)	51388	98525	255	710
8	60751	111329	255	774

ElementWise Multiply

The ElementWise Multiply calculates the Hadamard product of two input feature maps. The input channel of ElementWise Multiply ranges from 1 to 256 * channel_parallel. This feature is always enabled.

AveragePool

The AveragePool option determines whether the average pooling operation

will be performed on the DPUCZDX8G or not. This feature is always enabled, and supports rectangle sizes.

ReLU Type

The ReLU Type option determines the specific ReLU activation functions that can be used with the DPUCZDX8G. ReLU and ReLU6 are supported by default. Selection of the option “ReLU + LeakyReLU + ReLU6” will enable LeakyReLU as an activation function.

 **Note:** For this implementation, the LeakyReLU coefficient is fixed to 0.1 which is approximated as $26/256 = 0.1015625$

Table: Extra LUTs with ReLU + LeakyReLU + ReLU6 compared to ReLU+ReLU6

DPUCZDX8G Architecture	Extra LUTs from Conv	Extra LUTs from ALU
B512	465	493
B800	502	747
B1024	636	643
B1152	1038	896
B1600	831	658
B2304	460	439
B3136	812	1051
B4096	746	947

Argmax and Max

The Save Argmax option enables argmax and max feature along channel dimension when restoring the outputs back to DDR space. In some cases

like segmentation, only the index of the maximum value is needed. Then it is useful to replace softmax with argmax in the model to remove Exp calculation and reduce latency.

Table: Extra Resources with Save Argmax.

DPUCZDX8G Architecture	Extra LUTs	Extra Registers
B512	422	556
B800	399	547
B1024	460	546
B1152	503	631
B1600	590	640
B2304	803	442
B3136	832	758
B4096	735	389

Softmax

This option enables a hardware implementation of the softmax operator. The hardware softmax accelerator is packaged inside the DPU IP wrapper, but is a separate accelerator with its own interface and runtime and implements an int8 input and floating-point output data formats. The hardware implementation of softmax can be up to 160 times faster than a software implementation on MPSoC devices. Users can enable this option if their networks/models include a softmax layer and they wish to improve throughput.

Note: The hardware softmax can support up to 1023 classes. If the number of classes is greater than 1023, a software implementation of softmax may be considered. For more information, refer to the Vitis AI Library User Guide ([UG1354](#)) .

When hardware softmax is enabled, an AXI master interface named SFM_M_AXI and an interrupt port named `sfm_interrupt` are added to the DPU IP wrapper. The softmax module uses `m_axi_dpu_aclk` as the source clock for SFM_M_AXI as well as for computation.

The additional resources utilized when hardware softmax acceleration is enabled are listed in the following table.

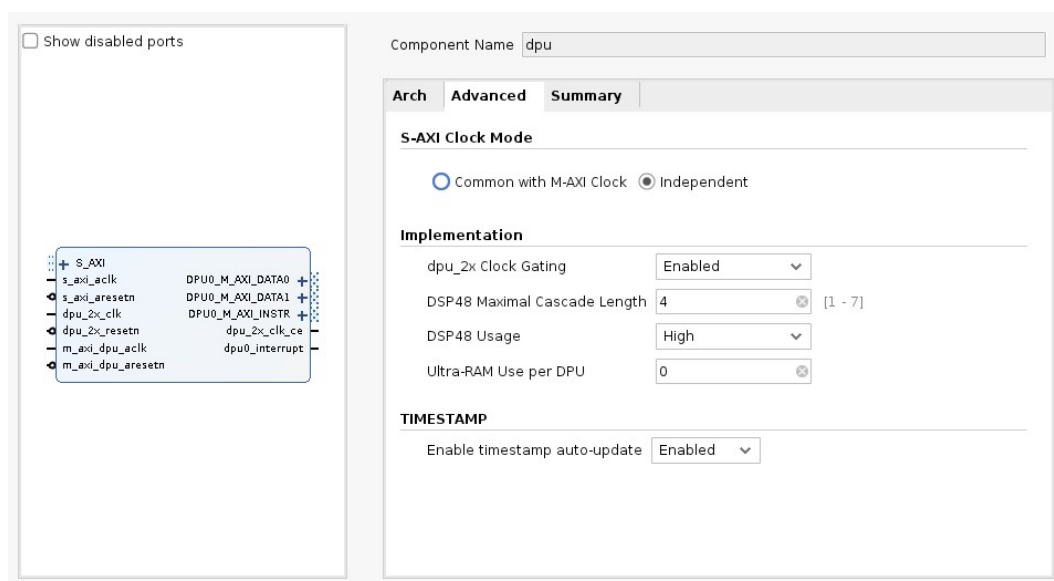
Table: Extra Resources with Softmax

IP Name	Extra LUTs	Extra FFs	Extra BRAMs	Extra DSPs
Softmax	9580	8019	4	14

Advanced Tab

The following figure shows the Advanced tab of the DPUCZDX8G configuration.

Figure: DPUCZDX8G Configuration – Advanced Tab



- S-AXI Clock Mode

`s_axi_aclk` is the S-AXI interface clock. When Common with M-AXI Clock is selected,

`s_axi_aclk` shares the same clock as `m_axi_aclk` and the `s_axi_aclk` port is hidden. When Independent is selected, a clock different from `m_axi_aclk` must be provided.

- **dpu_2x Clock Gating**

`dpu_2x` clock gating is an option for reducing the power consumption of the DPUCZDX8G. When the option is enabled, a port named `dpu_2x_clk_ce` appears for each DPUCZDX8G core. The `dpu_2x_clk_ce` port should be connected to the `clk_dsp_ce` port in the `dpu_clk_wiz` IP. The `dpu_2x_clk_ce` signal can shut down the `dpu_2x_clk` when the computing engine in the DPUCZDX8G is idle. To generate the `clk_dsp_ce` port in the `dpu_clk_wiz` IP, the clocking wizard IP should be configured with specific options. For more information, see the [Reference Clock Generation](#) section.

- **DSP Cascade**

The maximum length of the DSP48E slice cascade chain can be set. Longer cascade lengths typically use fewer logic resources but might have worse timing. Shorter cascade lengths might not be suitable for small devices as they require more hardware resources. Xilinx recommends selecting the mid-value, which is four, in the first iteration and adjust the value if the timing is not met.

- **DSP Usage**

This allows you to select whether DSP48E slices will be used for accumulation in the DPUCZDX8G convolution module. When low DSP usage is selected, the DPUCZDX8G IP will use DSP slices only for multiplication in the convolution module. In high DSP usage mode, the DSP slice will be used for both multiplication and accumulation. Thus, the high DSP usage consumes more DSP slices and fewer LUTs. The additional logic resources required for High versus Low DSP usage is shown in the following table:

Table: Extra Resource Utilization of High DSP Compared to Low DSP Usage

DPUCZDX8G Architecture	Extra LUTs	Extra Registers	Extra DSPs ¹
B512	1418	1903	-32
B800	1445	2550	-40
B1024	1978	3457	-64
B1152	1661	2525	-48
B1600	2515	4652	-80
B2304	3069	4762	-96
B3136	3520	6219	-112
B4096	3900	7359	-128
1. Negative numbers imply a relative decrease.			

- **UltraRAM**

There are two kinds of on-chip memory resources in Zynq® UltraScale+™ devices: block RAM and UltraRAM. The available amount of each memory type is device-dependent. Each block RAM consists of two 18K slices which can be configured as 9b*4096, 18b*2048, or 36b*1024. UltraRAM has a fixed-configuration of 72b*4096. A memory unit in the DPUCZDX8G has a width of ICP*8 bits and a depth of 2048. For the B1024 architecture, the ICP is 8, and the width of a memory unit is 8*8 bit. Each memory unit can then be instantiated with one UltraRAM block. When the ICP is greater than eight, each memory unit in the DPUCZDX8G needs at least two UltraRAM blocks.

The DPUCZDX8G uses block RAM as the memory unit by default. For a target device with both block RAM and UltraRAM, configure the number of UltraRAM to determine how many UltraRAMs are used to replace some block RAMs. The number of UltraRAM should be set as a multiple of the number of UltraRAM required for a memory unit in the DPUCZDX8G. An example of block RAM and UltraRAM usage is shown in the Summary tab section.

- **Timestamp**

When enabled, the DPUCZDX8G records the time that the DPUCZDX8G project was synthesized. When disabled, the timestamp keeps the value at the moment of the last IP update.

Related Information

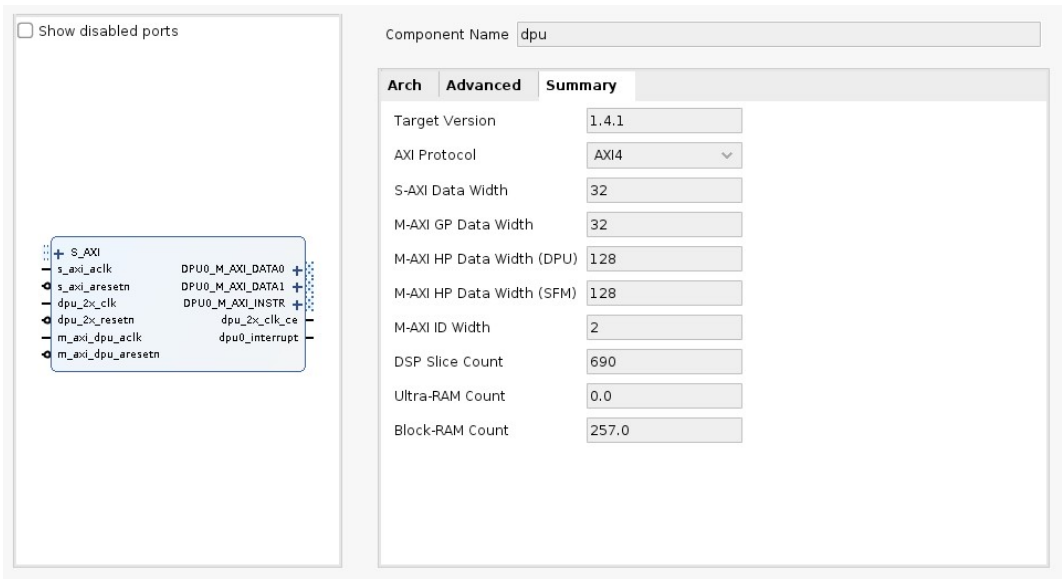
[Reference Clock Generation](#)

[Summary Tab](#)

Summary Tab

A summary of the configuration settings is displayed in the Summary tab. The Target Version field denotes the DPUCZDX8G instruction set version number.

Figure: DPUCZDX8G Configuration – Summary Tab



DPUCZDX8G Peak Performance

The following table shows the peak theoretical performance of the DPUCZDX8G:

Table: DPUCZDX8G Performance GOPs per second (GOPS)

Device	DPUCZDX8G Configuration	Frequency (MHz)	Peak Theoretical Performance (GOPS)
ZU2	B1152x1	370	426
ZU3	B2304x1	370	852
ZU5	B4096x1	350	1400
ZU7EV	B4096x2	330	2700
ZU9	B4096x3	333	4100
1. GOPS = PP*ICP*OCP*2 * Freq			

Xilinx Model Zoo Performance

In this section, the performance of several models is provided for reference. The results shown in the following table were measured on a Xilinx® ZCU102 board with three B4096 cores with 6 threads running at 287 MHz. A complete listing of Xilinx® Model Zoo performance for the ZCU102, ZCU104 and Kria KV260 is available here: https://github.com/Xilinx/Vitis-AI/tree/master/model_zoo#model-performance

Table: Performance of Different Models

Network Model	Workload (GOPs per image)	Input Image Resolution	Accuracy (DPIIC7DX8 G) ²	Frames per second (FPS)
Inception-v1	3.16	224*224	Top-1: 0.6984	472.5
ResNet50	7.7	224*224	Top-1: 0.7334	194.1
MobileNet_v2	0.59	224*224	Top-1: 0.6349	747.3
SSD_ADAS_VEHICLE ¹	6.3	480*360	mAP: 0.4261	297
SSD_ADAS_P EDESTRIAN ¹	5.9	640*360	mAP: 0.5968	278.3
SSD_Mobile Net_v2	6.57	480*360	mAP: 0.2931	113.3
YOLO-V3-VOC	65.42	416*416	mAP: 0.8127	34.7
YOLO-V3_ADAS ¹	5.46	512*256	mAP: 0.5305	272.6

Network Model	Workload (GOPs per image)	Input Image Resolution	Accuracy (DPU C7DX8 G) 2	Frames per second (FPS)
---------------	---------------------------	------------------------	--------------------------	-------------------------

1. These models were pruned by Vitis AI Optimizer.
2. Accuracy values with 8-bit quantization.

Unsupported Models

Some models found in the Xilinx Model Zoo may not be supported on smaller DPU architectures due to feature map size limitations. The following list specifies the models that are not supported on these smaller DPU architectures. The B4096 supports all models listed in the ZCU102 model-performance listings on GitHub: https://github.com/Xilinx/Vitis-AI/tree/master/model_zoo#model-performance.

Table: Unsupported Models by Architecture

DPUCZDX8G Architecture	Unsupported models
B512	inception_resnet_v2_tf
	vgg_16_tf
	vgg_19_tf
	facerec_resnet20
	facerec_resnet64
	facerec-resnet20_mixed_pt
	efficientnet-b0_tf2
	efficientdet_d2_tf
	ocr_pt
	textmountain_pt

DPUCZDX8G Architecture	Unsupported models
B800	SA_gate_base_pt
	HRNet_pt
	vgg_16_tf
	vgg_19_tf
	facerec_resnet20
	facerec_resnet64
	facerec-resnet20_mixed_pt
	efficientnet-b0_tf2
	efficientdet_d2_tf
	ocr_pt
	textmountain_pt
	SA_gate_base_pt
B1024	inception_resnet_v2_tf
	vgg_16_tf
	vgg_19_tf
	facerec_resnet20
	facerec_resnet64
	facerec-resnet20_mixed_pt
	efficientdet_d2_tf
	ocr_pt
	textmountain_pt
	SA_gate_base_pt
B1152	vgg_16_tf

DPUCZDX8G Architecture	Unsupported models
	vgg_19_tf
	efficientnet-b0_tf2
	ocr_pt
	SA_gate_base_pt
B1600	vgg_16_tf
	vgg_19_tf
	facerec_resnet20
	facerec_resnet64
	facerec-resnet20_mixed_pt
	textmountain_pt
	SA_gate_base_pt
B2304	vgg_16_tf
	vgg_19_tf
	SA_gate_base_pt
B3136	vgg_16_tf
	vgg_19_tf
	SA_gate_base_pt

I/O Bandwidth Requirements

When different neural networks run on the DPUCZDX8G, the I/O bandwidth requirement will vary depending on which neural network is currently being executed. Even the I/O bandwidth requirement of different layers in a single neural network will be different. The I/O bandwidth requirements for some neural networks, averaged by layer, have been captured with one DPUCZDX8G core running at full speed. The peak and average I/O

bandwidth requirements of three different neural networks are shown in the table below. The data provided is for two commonly used DPUCZDX8G architectures (B1152 and B4096).

 **Note:** When multiple DPUCZDX8G cores run in parallel, each core might not be able to achieve optimum performance due to I/O bandwidth limitations.

Table: I/O Bandwidth Requirements for B1152 and B4096

Network Model	B1152		B4096	
	Peak (MB/s)	Average (MB/s)	Peak (MB/s)	Average (MB/s)
Inception-v1	1704	890	4626	2474
ResNet50	2052	1017	5298	3132
SSD ADAS VEHICLE	1516	684	5724	2049
YOLO-V3-VOC	2076	986	6453	3290

If one DPUCZDX8G core needs to run at full speed, the peak I/O bandwidth requirement shall be met. These bandwidth requirements are primarily associated access to data though the AXI master interfaces (DPU0_M_AXI_DATA0 and DPU0_M_AXI_DATA1).

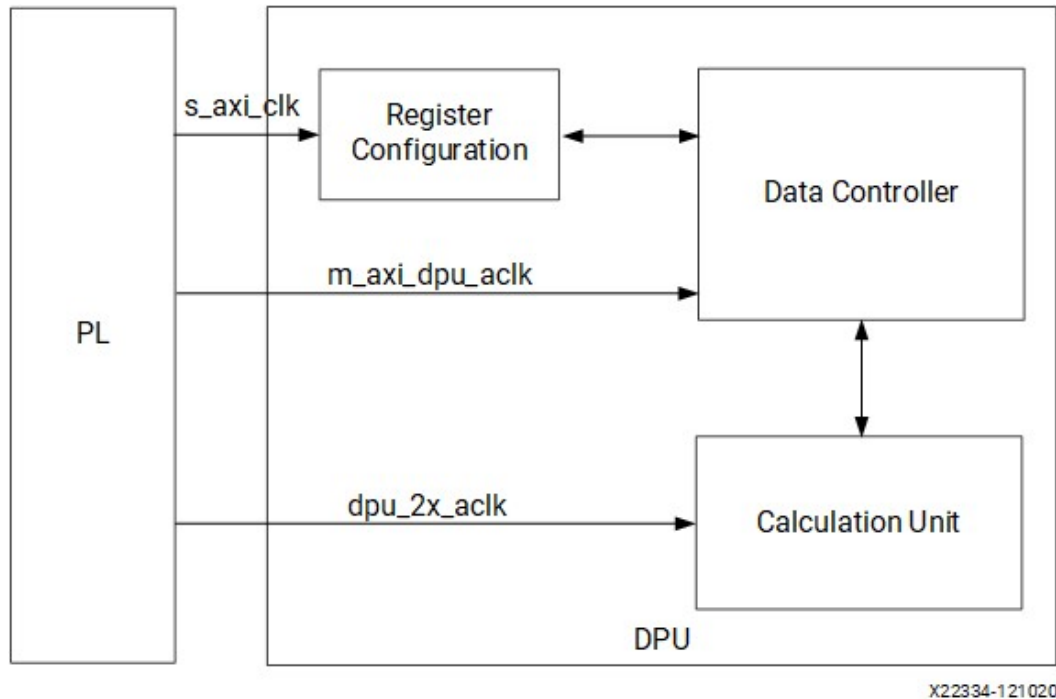
Clocking

There are three clock domains in the DPUCZDX8G IP, associated with register configuration, the DPU data plane and computation. The three input clocks can be configured independently in order to match system and performance requirements. Therefore, the corresponding reset for the three input clocks must be configured correctly.

Clock Domain

The following figure shows the three clock domains.

Figure: Clock Domains in the DPUCZDX8G



Register Clock

`s_axi_aclk` is used for the register configuration module. This module receives the DPUCZDX8G configuration through the S_AXI interface. The S_AXI clock can be common with the M-AXI clock or an independent clock. The DPUCZDX8G configuration registers are updated at a very low frequency and most of those registers are set only at the start of a task. Because the M-AXI domain directly impacts DPU performance, Xilinx recommends that the user configure the S-AXI is an independent clock with a frequency of 100MHz, while targeting a higher frequency for the M-AXI clocks.

In the Vitis flow, the platform may provide only two clocks for the DPUCZDX8G IP. In this case, the S_AXI clock must be configured as common

with the M-AXI clock and target the higher frequency.

Data Controller Clock

The primary function of the data controller module is to schedule the data flow in the DPUCZDX8G IP. The data controller module is clocked from `m_axi_dpu_aclk`. Data transfer between the DPUCZDX8G and external memory is clocked from the data controller clock domain, thus, `m_axi_dpu_aclk` serves as a clock source for both the data controller and the AXI_MM master interface. `m_axi_dpu_aclk` should be connected to the AXI_MM master clock.

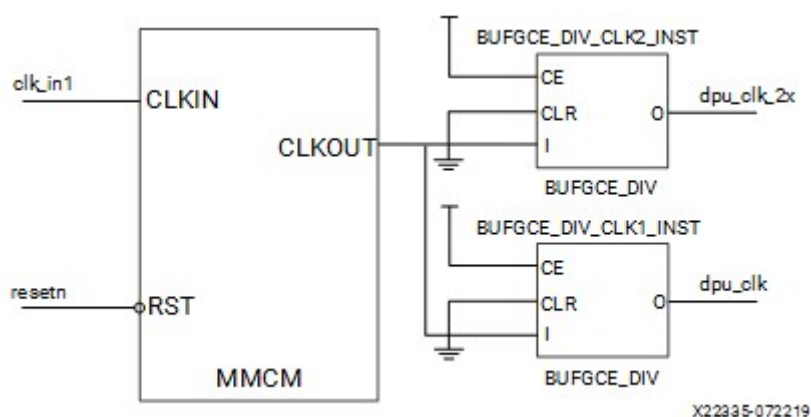
Computation Clock

The DSP slices in the computation unit module are in the `dpu_2x_clk` domain, which runs at twice the clock frequency of the data controller module. The two related clocks must be edge-aligned.

Reference Clock Generation

The `m_axi_dpu_aclk` (1x) and `dpu_2x_clk` clock must be synchronous and edge-aligned. While there are various techniques available to achieve this, the recommended clock topology is illustrated in the figure below:

Figure: Reference Circuit



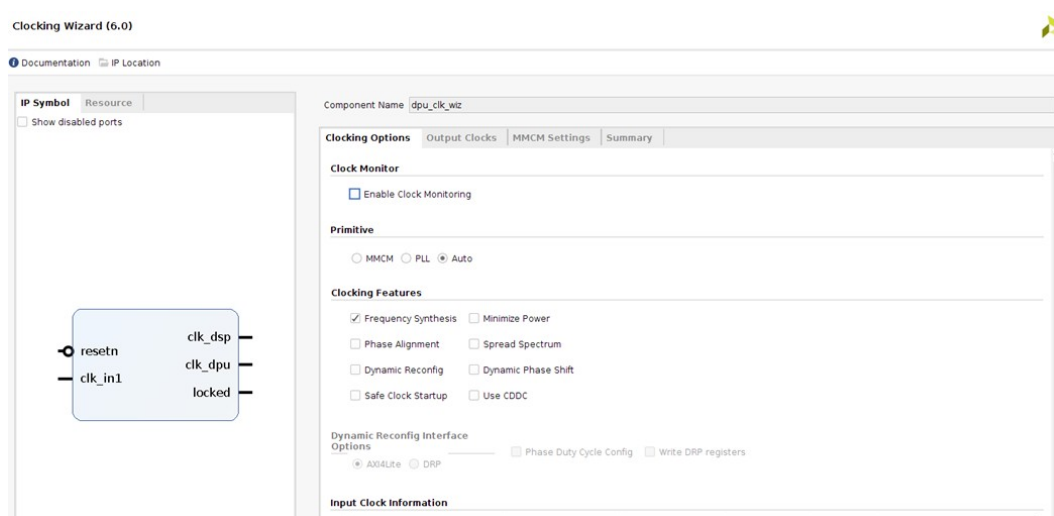
As shown, MMCM and two BUFGCE_DIV blocks are instantiated to generate the 1x and 2x clocks. The frequency of `clk_in1` is arbitrary and the frequency of MMCMCLKOUT should be selected to be equal to the target frequency of `dpu_clk_2x` (BUFGCE_DIV_CLK2_INST BUFGCE_DIVIDE = 1). BUFGCE_DIV_CLK1_INST in turn is configured to divide the MMCM output clock frequency by two (BUFGCE_DIVIDE=2). The resulting two output clocks are synchronous and phase aligned. Dedicated routing from the MMCM to BUFGCE_DIV instances ensures that skew between the two clocks is minimized.

Configuring Clock Wizard

An instance of the Xilinx clock wizard IP can implement the above circuit. The following example is provided to illustrate how the user can implement such a design. In this example, the frequency of `s_axi_aclk` is set to 100 MHz and `m_axi_dpu_aclk` is set to 325 MHz. Therefore, the frequency of the `dpu_2x_clk` should be set to 650 MHz accordingly. The recommended configuration of the Clocking Options tab is shown in the following figure.

 **Note:** The primitive parameter radio button must be set to Auto.

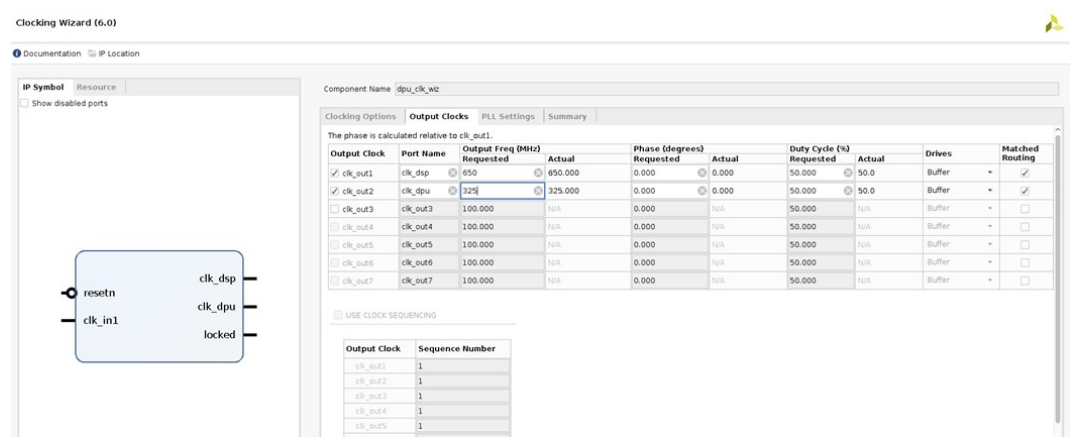
Figure: Recommended Clock Wizard Clocking Options



In addition, Matched Routing must be selected for `m_axi_dpu_aclk` and `dpu_2x_clk` in the Output Clocks tab of the Clock Wizard IP. The Matched

Routing option ensures that the skew between two or more output clocks from the Clock Wizard IP (in this case, the output of the two BUFGCE_DIV instances) is minimized. The related configuration is shown in the following figure.

Figure: Matched Routing in Clock Wizard



Note: When populating the Output Clocks tab, populate the clocks from the highest frequency to the lowest. This ensures that the architecture implemented leverages the BUFGCE_DIV. The following figure shows the correct sequence. The settings in figure (a) achieved the dedicated clock design in the Summary page while the figure (b) did not. For instance, we find that the frequency ordering used in Figure 18(a) generates two BUFGCE_DIV buffers, while the incorrect ordering used in Figure 18(b) will produce an instantiation that leverages two discrete outputs from the PLL or MMCM, with the result that the output skew is non-ideal. For more details, refer to the Clocking Wizard LogiCORE IP Product Guide ([PG065](#)).

Figure: Comparison of clkout Frequency Sequence

Clocking Options

Output Clocks

PLL Settings

Summary

The phase is calculated relative to clk_out1.

Output Clock	Port Name	Output Freq (MHz) Requested	Actual
☒ clk_out1	clk_dsp	650	650.00000
☒ clk_out2	clk_dpu	325	325.00000

Clocking Options

Output Clocks

PLL Settings

Summary

Clocking Primitive Attributes

Primitive Instantiated : PLL

Divide Counter : 1

Mult Counter : 13

Clock Phase Shift : None

Clock Wiz O/p Pins	Source	Divider Value
clk_dsp	PLL CLKOUT0	2
clk_dpu	BUFGCE_DIV driven	2.0
clk_out3	OFF	OFF
clk_out4	OFF	OFF
clk_out5	OFF	OFF
clk_out6	OFF	OFF
clk_out7	OFF	OFF

(a)

Clocking Options

Output Clocks

PLL Settings

Summary

The phase is calculated relative to clk_out1.

Output Clock	Port Name	Output Freq (MHz) Requested	Actual
☒ clk_out1	clk_dpu	325	325.00000
☒ clk_out2	clk_dsp	650	650.00000

Clocking Options

Output Clocks

PLL Settings

Summary

Clocking Primitive Attributes

Primitive Instantiated : PLL

Divide Counter : 1

Mult Counter : 13

Clock Phase Shift : None

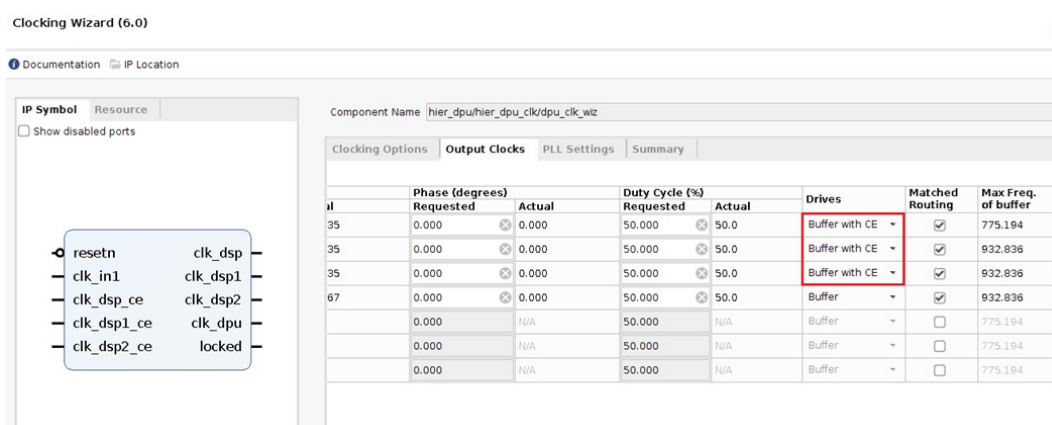
Clock Wiz O/p Pins	Source	Divider Value
clk_dpu	PLL CLKOUT0	4
clk_dsp	PLL CLKOUT1	2
clk_out3	OFF	OFF
clk_out4	OFF	OFF
clk_out5	OFF	OFF
clk_out6	OFF	OFF
clk_out7	OFF	OFF

(b)

Adding CE for dpu_2x_clk

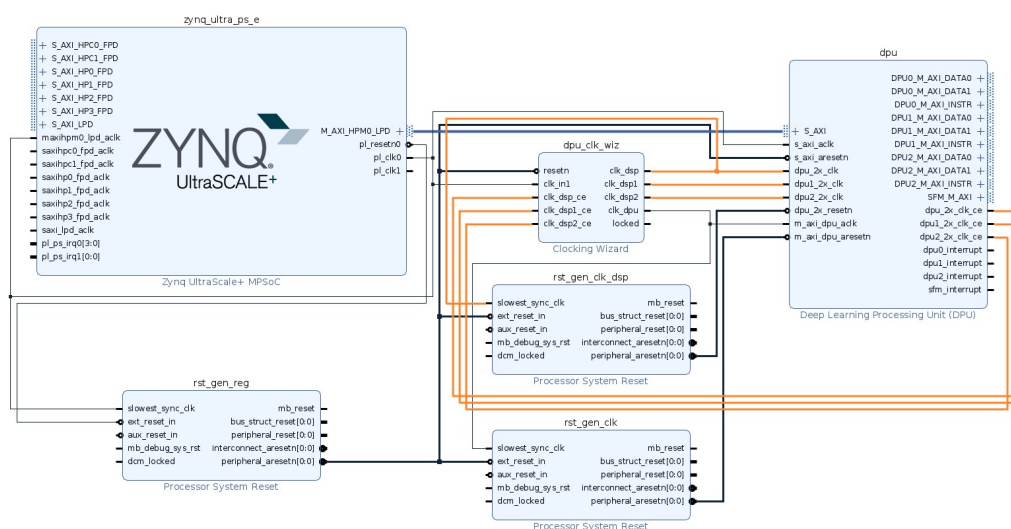
The `dpu_2x` clock gating option can reduce the power consumption of the DPUCZDX8G. When the option is enabled, the number of generated `clk_dsp` should be equal to the number of DPUCZDX8G cores. Each `clk_dsp` should be set as a buffer with CE in the clock wizard IP. As shown in the following figure, three `clk_dsp_ce` appear when the output clock is configured with the CE. To enable the `dpu_2x` clock gating function, each `clk_dsp_ce` port should be connected to the corresponding `dpu_2x_clk_ce` port in the DPUCZDX8G.

Figure: Configure Clock Wizard with Buffer CE



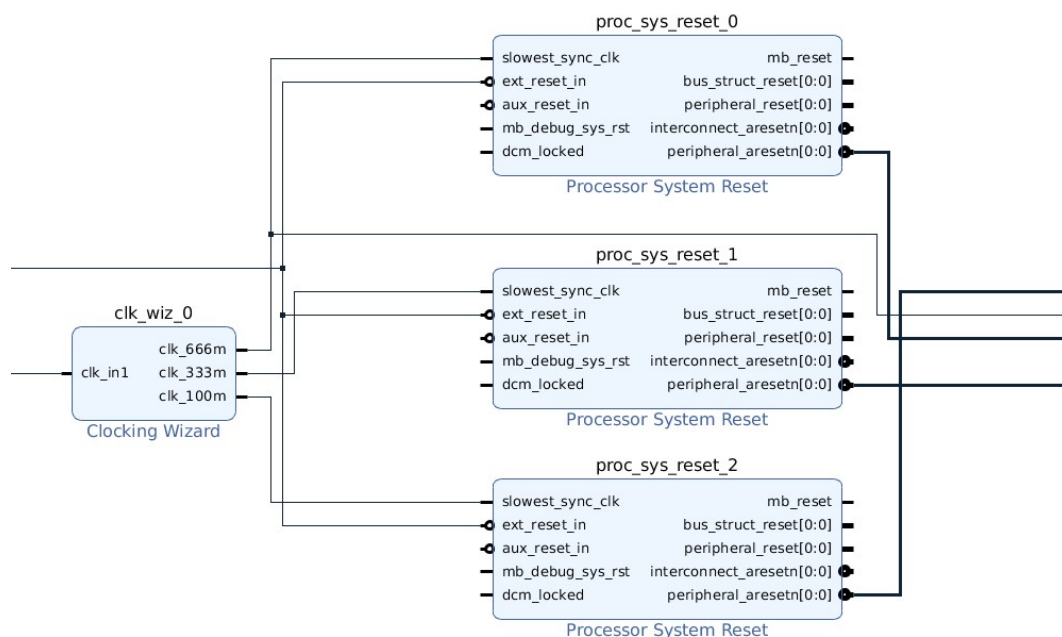
After configuring the clock wizard, the `clk_dsp_ce` should be connected to the corresponding port in the DPUCZDX8G. The connections are shown in the following figure.

Figure: Clock CE and DPUCZDX8G Connections



Reset

There are three input clocks for the DPUCZDX8G IP and each clock has a corresponding reset. Each reset must be synchronous to its corresponding clock. If the related clocks and resets are not synchronized, correct operation of the DPU cannot be guaranteed. A Processor System Reset IP block is recommended to generate a synchronized reset signal. The reference design is shown here.

Figure: DPU Reset Circuit Example

Development Flow

Customizing and Generating the Core in the Zynq UltraScale+ MPSoC

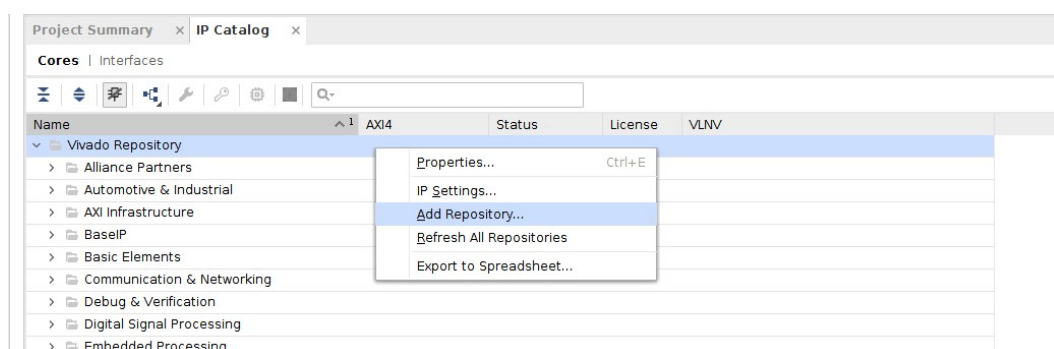
The following sections describe the development flow on how to use the DPUCZDX8G IP with the Vivado® Design Suite:

- Add the DPUCZDX8G into Repository or Upgrade the DPUCZDX8G from a Previous Version
- Add the DPUCZDX8G into the Block Design
- Configure DPUCZDX8G Parameters
- Connecting the DPUCZDX8G to the Processing System in the Zynq UltraScale+ MPSoC
- Assign Register Addresses
- Generate Bitstream
- Generate BOOT.BIN

Add the DPUCZDX8G into Repository or Upgrade the DPUCZDX8G from a Previous Version

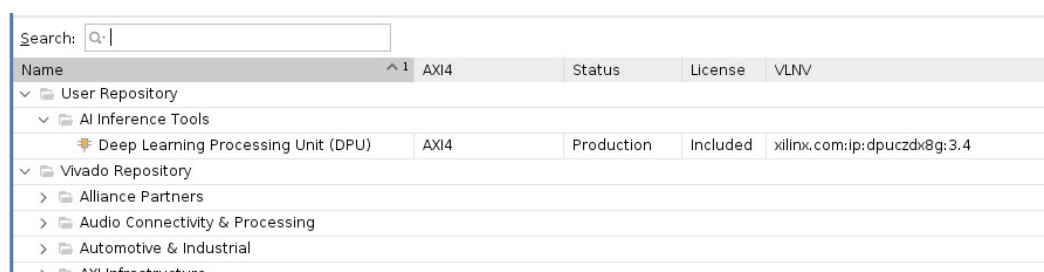
In the Vivado Integrated Design Environment (IDE), click Project Manager > IP Catalog. In the IP catalog tab, right-click and select Add Repository (see figure below), then select the location of the DPUCZDX8G IP.

Figure: Add Repository



The DPUCZDX8G IP will appear in the IP catalog page.

Figure: DPUCZDX8G IP in Repository



If there is an existing hardware project with an old version of the DPUCZDX8G, upgrading to the latest version is required, follow these steps:

1. Delete the old DPUCZDX8G IP in the block design and IP repository.
2. Add the new DPUCZDX8G IP into the IP repository.
3. Add the new DPUCZDX8G IP into the block design.

Add the DPUCZDX8G into the Block Design

Search for DPUCZDX8G in the block design interface and add the DPUCZDX8G IP into the block design. The procedure is shown in the following figures.

Figure: Search DPUCZDX8G IP

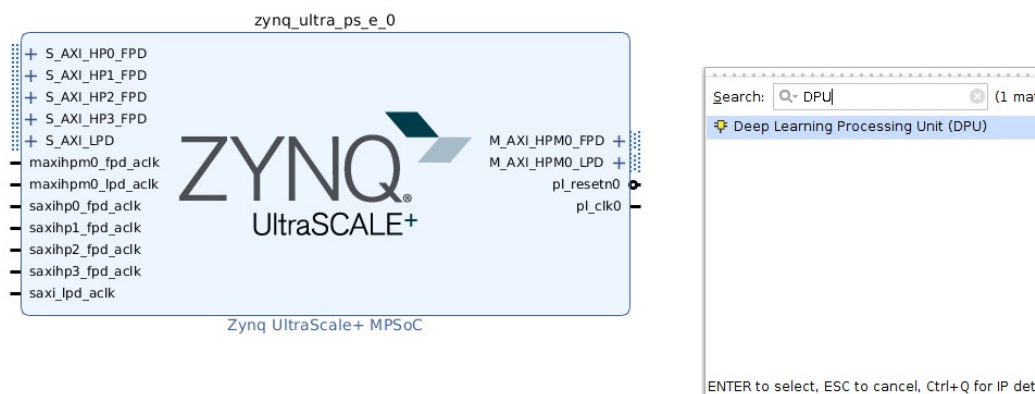
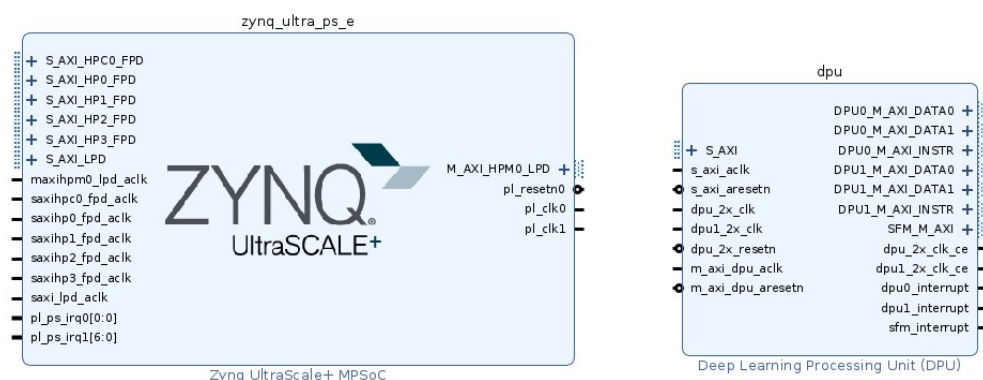


Figure: DPUCZDX8G IP into Block Design



Configure DPUCZDX8G Parameters

You can configure the DPUCZDX8G IP as shown in the settings figure. Details about these parameters can be found in the [DPU Configuration](#) section.

Connecting the DPUCZDX8G to the Processing System in the Zynq UltraScale+ MPSoC

The DPUCZDX8G IP contains only one slave interface. The number of DPUCZDX8G cores depends on the parameter DPU_NUM, which is configured through the Number of DPU Cores option in the wizard. Each DPUCZDX8G core has three master interfaces, one for instruction fetch, and the other two for data access.

The DPUCZDX8G IP can be connected to the processing system (PS) with an AXI Interconnection IP as long as the DPUCZDX8G can correctly access the DDR memory space. Generally, when data is transferred through an Interconnect IP, the data transaction delay will increase. The delay incurred

by the Interconnect will reduce the DPUCZDX8G performance. Therefore, Xilinx recommends that each master interface in the DPUCZDX8G is connected to the PS through a direct connection rather than through an AXI Interconnect IP when there are sufficient AXI slave ports available on the PS.

When the AXI slave ports of the PS are insufficient for the DPUCZDX8G, an AXI interconnect for connection is unavoidable. The two AXI master ports for data fetching are high bandwidth ports and the AXI master port for instruction fetching is a low bandwidth port. Typically, it is recommended that all the master ports for instruction fetching connect to the S_AXI_LPD of PS through one interconnect. The rest of the master ports for data fetching should be directly connected to the PS as much as possible. Xilinx recommends that the master ports of the DPUCZDX8G core with higher priority (smaller number, like DPU0) be directly connected to the slave ports of the PS with higher priority (smaller number, like S_AXI_HP0_FPD).

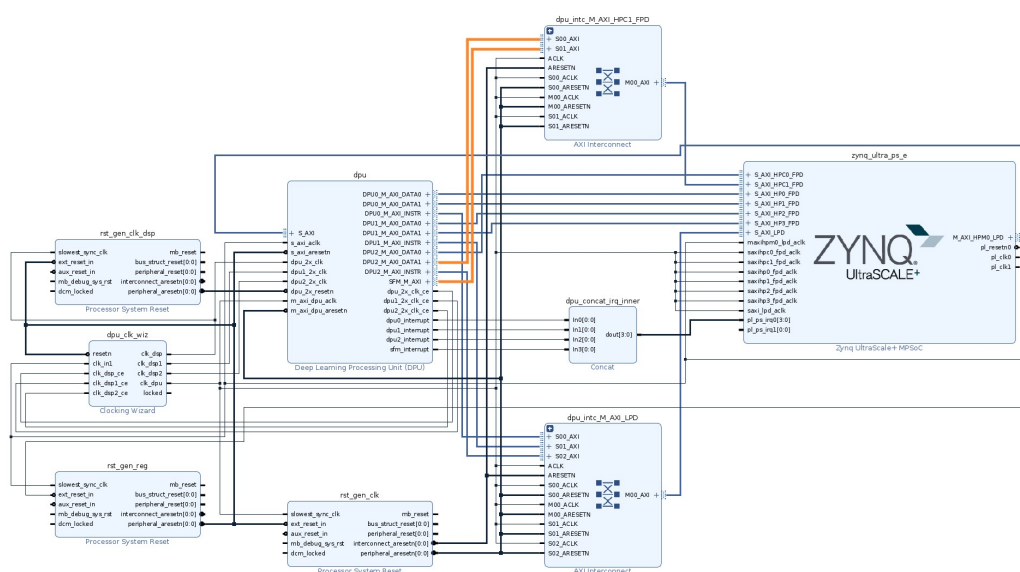
For example, if there are three DPUCZDX8G cores and one SFM core, and there are only seven master ports, and four slave ports: S_AXI_HP1~3 and S_AXI_HPC0. A possible connection setup would be:

- DPU0_M_AXI_DATA0 to HP1
- DPU0_M_AXI_DATA1 to HP2
- DPU1_M_AXI_DATA0 and DPU1_M_AXI_DATA1 to HP3
- DPU2_M_AXI_DATA0, DPU2_M_AXI_DATA1, and SFM to HPC0

It is recommended that the slave port of DPUCZDX8G be connected to M_AXI_HPM0_LPD of the PS.

A reference connection between the DPUCZDX8G and PS in the Zynq UltraScale+ MPSoC is shown here. The number of DPUCZDX8G core is set to three, and the Softmax function is enabled.

Figure: DPUCZDX8G and PS Connections for Zynq UltraScale+ MPSoC



Assign Register Addresses

When the DPUCZDX8G connection is complete, the next step is to assign the register address of the AXI slave interface. The minimum space needed for the DPUCZDX8G is 16 MB. The DPUCZDX8G slave interface can be assigned to any starting address accessible by the host CPU.

 **Note:** The DPUCZDX8G base address must be set with a range of 16 MB. The DPUCZDX8G only uses LPD_DDR_LOW or LPD_DDR_HIGH segment. As for LOD_DDR_HIGH segment, it depends on the system memory of the board is beyond 2GB[0 – 0x7fff_ffff and 0x8000_0000 –]. The LPD_DDR_LOW or LPD_DDR_HIGH is decided by the Linux kernel configuration. The DPUCZDX8G will use the memory in the LPD_DDR_LOW segment by default.

The reference address assignments of the DPUCZDX8G are shown here.

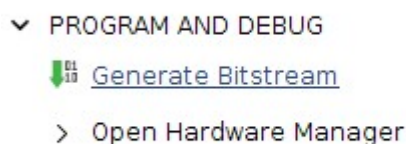
Figure: DPUCZDX8G Address Assignment

Cell	Slave Interface	Base Name	Offset Address	Range	High Address
zynq_ultra_ps_e_0					
Data (40 address bits : 0x00A0000000 [256M] ,0x0400000000 [4G] ,0x1000000000 [224G] ,0x0080000000 [512M])					
dpu_eu_0	S_AXI	reg0	0x00_8F00_0000	16M	0x00_8FFF_FFFF
dpu_eu_0					
DPU0_M_AXI_GP0 (40 address bits : 1T)					
zynq_ultra_ps_e_0	S_AXI_LPD	LPD_DDR_LOW	0x00_0000_0000	2G	0x00_7FFF_FFFF
zynq_ultra_ps_e_0	S_AXI_LPD	LPD_LPS_OCM	0x00_FF00_0000	16M	0x00_FFFF_FFFF
DPU0_M_AXI_HP0 (40 address bits : 1T)					
zynq_ultra_ps_e_0	S_AXI_HP0_FPD	HP0_DDR_LOW	0x00_0000_0000	2G	0x00_7FFF_FFFF
zynq_ultra_ps_e_0	S_AXI_HP0_FPD	HP0_LPS_OCM	0x00_FF00_0000	16M	0x00_FFFF_FFFF
DPU0_M_AXI_HP1 (40 address bits : 1T)					
zynq_ultra_ps_e_0	S_AXI_HP1_FPD	HP1_DDR_LOW	0x00_0000_0000	2G	0x00_7FFF_FFFF
zynq_ultra_ps_e_0	S_AXI_HP1_FPD	HP1_LPS_OCM	0x00_FF00_0000	16M	0x00_FFFF_FFFF
DPU1_M_AXI_GP0 (40 address bits : 1T)					
zynq_ultra_ps_e_0	S_AXI_LPD	LPD_DDR_LOW	0x00_0000_0000	2G	0x00_7FFF_FFFF
zynq_ultra_ps_e_0	S_AXI_LPD	LPD_LPS_OCM	0x00_FF00_0000	16M	0x00_FFFF_FFFF
DPU1_M_AXI_HP0 (40 address bits : 1T)					
zynq_ultra_ps_e_0	S_AXI_HP2_FPD	HP2_DDR_LOW	0x00_0000_0000	2G	0x00_7FFF_FFFF
zynq_ultra_ps_e_0	S_AXI_HP2_FPD	HP2_LPS_OCM	0x00_FF00_0000	16M	0x00_FFFF_FFFF
DPU1_M_AXI_HP1 (40 address bits : 1T)					
zynq_ultra_ps_e_0	S_AXI_HP3_FPD	HP3_DDR_LOW	0x00_0000_0000	2G	0x00_7FFF_FFFF
zynq_ultra_ps_e_0	S_AXI_HP3_FPD	HP3_LPS_OCM	0x00_FF00_0000	16M	0x00_FFFF_FFFF

Generate Bitstream

Click Generate Bitstream in Vivado as shown below.

Figure: Generate Bitstream



Generate BOOT.BIN

To generate the BOOT.BIN file, use the Vivado® Design Suite or PetaLinux.

For create the boot image using the Vivado Design Suite, refer to the Zynq UltraScale+ MPSoC: Embedded Design Tutorial ([UG1209](#)). For PetaLinux, refer to the PetaLinux Tools Documentation: Reference Guide ([UG1144](#)).

Customizing and Generating the Core in the Vitis IDE

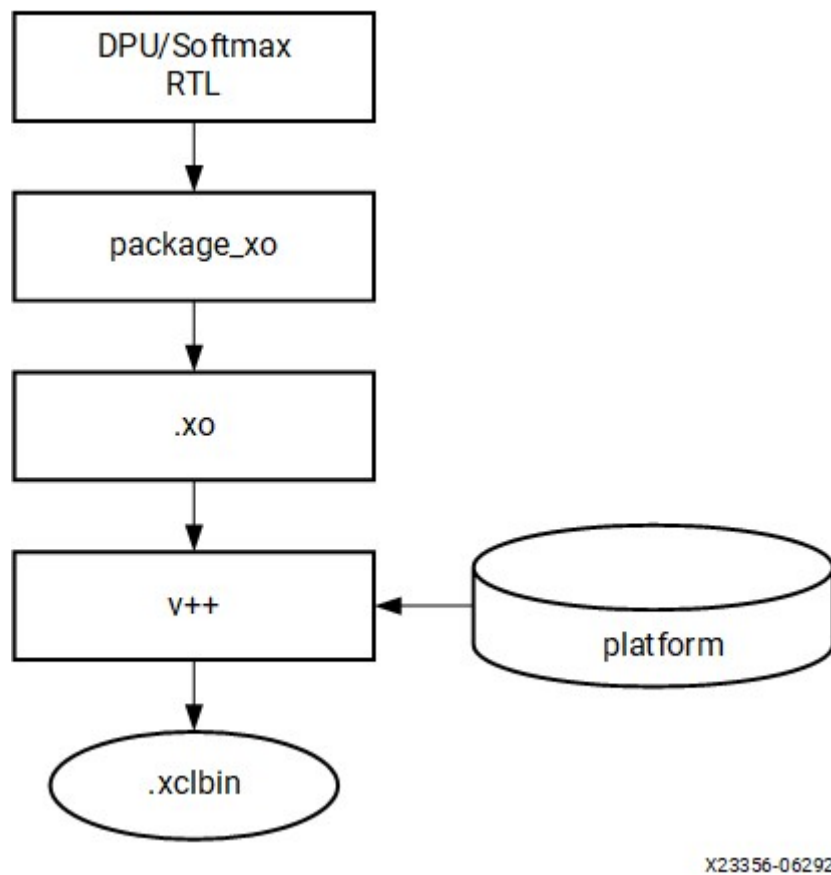
The Vitis™ embedded software development flow has the following two methods to generate the core:

1. GUI Flow
2. Command Flow

The command flow is recommended for its flexibility and ease of use. If you want to use the GUI flow, use the prj/Vitis#6-gui-flow flow here: <https://www.xilinx.com/bin/public/openDownload?filename=DPUCZDX8G.tar.gz>.

The build process for the DPUCZDX8G is shown in the following figure:

Figure: Build Process for the DPUCZDX8G



The following definitions describe the command flow and the process of using the DPUCZDX8G IP with the Vitis libraries:

- **Makefile and other scripts**

The DPUCZDX8G kernel is independently compiled to a Xilinx object (.xo) file. It is compiled using the package_xo utility. The RTL kernel wizard in the Vitis IDE can be used to simplify this process. The .xo file is linked with the hardware platform (shell) to create the FPGA binary (.xclbin). The v++ compiler automatically uses the Vivado® Design Suite tools to build the kernels to run on the FPGA platform.

The Makefile and the other scripts are present in the [Vitis DPU TRD](#).

- **Configure DPUCZDX8G Parameters**

You can modify the Vitis-AI/DPU-TRD/prj/Vitis/dpu_conf.vh file to configure the DPU parameters. See [Product Specification](#) for more details on the DPU parameters.

• Architecture

Select the hardware architecture from the following:: B512, B800, B1024, B1600, B2304, B3136, and B4096. For the B4096, the definition is as follows:

```
`define B4096
```

• UltraRAM Number

Modify the dpu_config.vh file to set the numbers. Enable ``define URAM_ENABLE` and ``define URAM_DISABLE`.

When UltraRAM is enabled, set the following parameters:

- `'define def_UBANK_IMG_N 5`
- `'define def_UBANK_WGT_N 17`
- `'define def_UBANK_BIAS 1`

The following table lists the recommended UltraRAM numbers for different architectures. You can also adjust the numbers according to the resource usage of the entire project.

Table: Recommended UltraRAM Numbers

	B512	B800	B1024	B1152	B1600	B2304	B3136	B4096
			4	2	0	4	6	6
U_BANK_IMG	2	2	4	2	4	4	4	5
U_BANK_WGT	9	11	9	13	11	13	15	17
U_BANK_BIAS	1	1	1	1	1	1	1	1

• RAM Usage

RAM usage high – ``define RAM_USAGE_HIGH`

RAM usage low – ``define RAM_USAGE_LOW`

- **Channel Augmentation**

Enable – ``define CHANNEL_AUGMENTATION_ENABLE`

Disable – ``define CHANNEL_AUGMENTATION_DISABLE`

- **ALU Parallel**

There are five options of ALU Parallel settings, they are:

- ALU_PARALLEL_DEFAULT
- ALU_PARALLEL_1
- ALU_PARALLEL_2
- ALU_PARALLEL_4
- ALU_PARALLEL_8

The ALU_PARALLEL_DEFAULT value is PP/2.

- **Conv RELU Type**

There are two options of RELU type, they are:

- CONV_RELU_RELU6
- CONV_RELU_LEAKYRELU_RELU6

If you want to use the RELU, Leaky ReLU, and ReLU6 of Conv, define as shown below:

```
`define CONV_RELU_LEAKYRELU_RELU6
```

- **ALU RELU Type**

There are two options of RELU type, they are:

- ALU_RELU_RELU6
- ALU_RELU_LEAKYRELU_RELU6

If you want to use the RELU, Leaky ReLU, and ReLU6 of ALU, define as shown below:

```
`define ALU_RELU_LEAKYRELU_RELU6
```

- **Argmax and Max**

Enable – ``define SAVE_ARGMAX_ENABLE`

Disable – ``define SAVE_ARGMAX_DISABLE`

- **DSP Usage**

High – ``define DSP48_USAGE_HIGH`

Low – ``define DSP48_USAGE_LOW`

- **Low Power Mode**

Enable – ``define LOWPOWER_ENABLE`

Disable – ``define LOWPOWER_DISABLE`

- **Device Configuration**

Support Zynq UltraScale+ MPSoC – ``define MPSOC.`

- **Set the DPUCZDX8G Number**

The number of cores is set to one by default. Add the [connectivity] property to configure the DPU number as follows:

```
[connectivity]
```

```
nk=dpu_xrt_top:2
```

The project will integrate two DPUs.

- **Softmax**

The Softmax IP is an independent kernel. The user can choose whether to integrate Softmax IP according to their own applications. Please refer to [Vitis DPU TRD Flow](#) for the detail method.

- **Specify Connectivity for the Ports**

Specify the connectivity to the various ports in the system for the DPU. Add the [connectivity] property to configure the DPUCZDX8G ports. Use the following command to check the ports of platform:

```
% platforminfo -p zcu102_base/zcu102_base.xpfm
```

Figure: Platform Information

```

=====
Hardware Platform (Shell) Information
=====
Vendor:                xilinx.com
Board:                 zcu102_base
Name:                  zcu102_base
Version:               1.0
Generated Version:     2019.2
Software Emulation:    1
Hardware Emulation:    0
FPGA Family:          zynqplus
FPGA Device:           xczu9eg
Board Vendor:          xilinx.com
Board Name:            xilinx.com:zcu102:3.3
Board Part:            xczu9eg-ffvb1156-2-e
Maximum Number of Compute Units: 60
=====
Clock Information
=====
Default Clock Index: 0
Clock Index:          0
Frequency:             149.985000
Clock Index:          1
Frequency:             299.970000
Clock Index:          2
Frequency:             74.992500
Clock Index:          3
Frequency:             99.990000
Clock Index:          4
Frequency:             199.980000
Clock Index:          5
Frequency:             399.960000
Clock Index:          6
Frequency:             599.940000
=====
Memory Information
=====
Bus SP Tag: HP0
Bus SP Tag: HP1
Bus SP Tag: HP2
Bus SP Tag: HP3
Bus SP Tag: HPC0
Bus SP Tag: HPC1
=====
Feature ROM Information
=====
=====
Software Platform Information
=====
Number of Runtimes:    1
Default System Configuration: zcu102_base
System Configurations:
System Config Name:    zcu102_base
System Config Description: zcu102_base
System Config Default Processor Group: xrt
System Config Default Boot Image: standard
System Config Is QEMU Supported: 1

```

If the platform does not have enough ports to connect to all the ports of the DPUCZDX8G, then the ports can be shared.

Add the [connectivity] property to specify the DPUCZDX8G ports as follows:

```

[connectivity]
sp=dpu_xrt_top_1.M_AXI_GP0:HP0
sp=dpu_xrt_top_1.M_AXI_HP0:HP1
sp=dpu_xrt_top_1.M_AXI_HP2:HP2
"

```

The project may have timing issues. You can add the [vivado] property to

configure the Vivado implementation strategy.

```
[vivado]  
prop=run.impl_1.strategy=Performance_Explore
```

The Vivado implementation step uses the Performance_Explore strategy.

Example Design

Introduction

The Xilinx® DPUCZDX8G targeted reference design (TRD) provides instructions on how to use the DPUCZDX8G with a Xilinx SoC platform to build and run deep neural network applications. The TRD includes two parts, the Vivado DPU TRD and the Vitis™ DPU TRD. The TRD uses the Vivado IP integrator flow for building the hardware design and the Xilinx Yocto PetaLinux flow for software design. The Zynq® UltraScale+™ MPSoC platform is used to create this TRD. The TRD can be accessed through this link: <https://github.com/Xilinx/Vitis-AI>.

This chapter describes the architecture of the reference design and provides a functional description of its components. It is organized as follows:

- Vivado TRD Overview provides a high-level overview of the Zynq UltraScale+ MPSoC architecture, the reference design architecture, and a summary of key features.
- Hardware Design gives an overview of how to use the Xilinx Vivado Design Suite to generate the reference hardware design.
- Software Design describes the design flow of project creation in the PetaLinux environment.
- Demo execution describes how to run an application created by the TRD.

The architecture of the Vitis DPU TRD reference design is similar to the Vivado TRD. However, the Vitis DPU TRD is friendly and flexible to the software designer.

Vivado DPU TRD Flow

For the Vivado DPU TRD Flow, refer to the <https://www.xilinx.com/bin/public/openDownload?filename=DPUCZDX8G.tar.gz>

This tutorial contains information about:

- Setting up the ZCU102 evaluation board and run the TRD.
- Changing the configuration of DPU.

Vitis DPU TRD Flow

For the Vitis™ DPU TRD Flow, refer to <https://www.xilinx.com/bin/public/openDownload?filename=DPUCZDX8G.tar.gz>

This tutorial contains information about:

- Setting up the ZCU102 evaluation board and running the TRD.
- Changing the DPU configuration.
- Integrating the DPU in a custom platform in the Vitis environment.
- Run the DPU_TRD in Vitis GUI flow.

Additional Resources and Legal Notices

Xilinx Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see [Xilinx Support](#).

Documentation Navigator and Design Hubs

Xilinx® Documentation Navigator (DocNav) provides access to Xilinx documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the Vivado® IDE, select Help > Documentation and Tutorials.
- On Windows, select Start > All Programs > Xilinx Design Tools > DocNav.
- At the Linux command prompt, enter `docnav`.

Xilinx Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and

address frequently asked questions. To access the Design Hubs:

- In DocNav, click the Design Hubs View tab.
- On the Xilinx website, see the [Design Hubs](#) page.

 **Note:** For more information on DocNav, see the [Documentation Navigator](#) page on the Xilinx website.

References

These documents provide supplemental material useful with this guide:

1. Vitis AI User Guide ([UG1414](#))
2. Zynq UltraScale+ MPSoC: Embedded Design Tutorial ([UG1209](#))
3. PetaLinux Tools Documentation: Reference Guide ([UG1144](#))
4. ZCU102 Evaluation Board User Guide ([UG1182](#))

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
01/23/2023 Version 4.1	
Entire document	• Added Correlation 1D and 2D • Added Argmax and Max along channel dimension • Optimized resources and timing
06/24/2022 Version 4.0	
Entire document	Made technical updates across the document for IP version upgrade to 4.0.
01/20/2022 Version 3.4	

Section	Revision Summary
Entire document	• Updated the interrupt signal of DPU image. • Deleted the device tree paragraph of Vivado flow. • Updated unsupported models table.
07/22/2021 Version 3.3	
DPU Configuration	• Added Unsupported Models. • Added a figure to illustrate the parallelism of DPU. • Added DPU resource table with UltraRAM enabled. • Support for Zynq-7000 devices has been discontinued. Contact your local sales representative.
02/03/2021 Version 3.3	
DPU Configuration	Updated Configuring Clock Wizard .
12/17/2020 Version 3.3	
Entire document	• Added Elementwise-Multiply function. • Added Max Reduce function. • Added constrain of Convolution and Deconvolution in Table 7. • Updated figures.
07/07/2020 Version 3.2	

Section	Revision Summary
Entire document	• Added Average Pool and BatchNormal in Table 7. • Added resources increment table of Average Pool, LeakyReLU, Depthwise conv, and Softmax . • Deleted the detailed description of Vivado flow and Vitis flow. For more details see the DPU_TRD GitHub.
03/23/2020 Version 3.2	
Entire document	• Updated the Vivado flow and Vitis flow for Target Version of 1.4.1. • Replaced the description of DNNDK and DNNC to Vitis AI and Vitis AI Compiler. • Updated the maximum of DPU core number from three to four and modified the descriptions accordingly.
12/02/2019 Version 3.1	
Entire document	Updated the flow for Vitis™ device support.
08/13/2019 Version 3.0	
Vitis AI Development Kit	Updated description.

Section	Revision Summary
Configuration Options	Added description in RAM Usage, Channel Augmentation, and updated numbers in Softmax section.
Advanced Tab	Added note in DSP Cascade and updated LUT numbers for High DSP in Resources for Different DSP Usage table.
Build the PetaLinux Project	Updated code.
07/31/2019 Version 3.0	
Overview	Updated whole chapter.
Product Specification	Updated whole chapter.
Table 1: DPU Signal Description	Added dpu_2x_clk_ce description.
DPU Configuration	Updated whole chapter.
Introduction	Updated description.
Table 7: Deep Neural Network Features and Parameters Supported by DPU	Updated Depthwise Convolution and Max Pooling descriptions.
Configuration Options	- Updated figures. - Added Channel Augmentation and dpu_2x Clock Gating sections and updated all description sections. - Updated Clocking and Reset.
Adding CE for dpu_2x_clk	Added section.
Development Flow	Updated whole chapter.

Section	Revision Summary
Add the DPUCZDX8G into Repository or Upgrade the DPUCZDX8G from a Previous Version	Updated section.
Customizing and Generating the Core in Zynq-7000 Devices	Updated figure.
Example Design	Updated whole chapter.
DPU Configuration	Updated section.
06/07/2019 Version 2.0	
Vitis AI Development Kit	Added description.
Table 1: DPU Signal Description	Added softmax descriptions.
Interrupts	Updated notes.
Table 7: Deep Neural Network Features and Parameters Supported by DPU	Added Depthwise Convolution.
Configuration Options	Added some new features: depthwise convolution, average pooling, ReLU type, softmax. Updated some figures of DPU GUI. Added description about s-axi clock mode.
Table 12: Performance of Different Models	Updated table.
Table 13: I/O Bandwidth Requirements for DPU-B1152 and DPU-B4096	Updated table.
Register Clock	Fixed the recommended frequency for DPU clock.

Section	Revision Summary
Configuring Clock Wizard	Updated description and figure.
Adding CE for dpu_2x_clk	Updated description and figure.
Configure DPUCZDX8G Parameters	Updated figure.
Connecting the DPUCZDX8G to the Processing System in the Zynq UltraScale+ MPSoC	Updated section.
Assign Register Addresses	Updated note.
Customizing and Generating the Core in Zynq-7000 Devices	Added section.
Design Files	Updated figure.
DPU Configuration	Updated figure.
Software Design	Updated section.
03/26/2019 Version 1.2	
Build the PetaLinux Project	Updated description.
Build the Demo	Updated figure.
Demo Execution	Updated code.
03/08/2019 Version 1.1	
Table 6: reg_dpu_base_addr	Updated description.
Figure 10: DPU Configuration	Updated figure.
Build the PetaLinux Project	Updated code.
Build the Demo	Updated description.
03/05/2019 Version 1.1	
Example Design	Added chapter regarding the DPU targeted reference design.

Section	Revision Summary
02/28/2019 Version 1.0	
Initial release.	N/A

Please Read: Important Legal Notices

The information disclosed to you hereunder (the "Materials") is provided solely for the selection and use of Xilinx products. To the maximum extent permitted by applicable law: (1) Materials are made available "AS IS" and with all faults, Xilinx hereby DISCLAIMS ALL WARRANTIES AND CONDITIONS, EXPRESS, IMPLIED, OR STATUTORY, INCLUDING BUT NOT LIMITED TO WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, OR FITNESS FOR ANY PARTICULAR PURPOSE; and (2) Xilinx shall not be liable (whether in contract or tort, including negligence, or under any other theory of liability) for any loss or damage of any kind or nature related to, arising under, or in connection with, the Materials (including your use of the Materials), including for any direct, indirect, special, incidental, or consequential loss or damage (including loss of data, profits, goodwill, or any type of loss or damage suffered as a result of any action brought by a third party) even if such damage or loss was reasonably foreseeable or Xilinx had been advised of the possibility of the same. Xilinx assumes no obligation to correct any errors contained in the Materials or to notify you of updates to the Materials or to product specifications. You may not reproduce, modify, distribute, or publicly display the Materials without prior written consent. Certain products are subject to the terms and conditions of Xilinx's limited warranty, please refer to Xilinx's Terms of Sale which can be viewed at <https://www.xilinx.com/legal.htm#tos>; IP cores may be subject to warranty and support terms contained in a license issued to you by Xilinx. Xilinx products are not designed or intended to be fail-safe or for use in any application requiring fail-safe performance; you assume sole risk and liability for use of Xilinx products in such critical applications, please refer to Xilinx's Terms of Sale which can be viewed at <https://www.xilinx.com/legal.htm#tos>.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2019–2023 Advanced Micro Devices, Inc. Xilinx, the Xilinx logo, Alveo, Artix, Kintex, Kria, Spartan, Versal, Vitis, Virtex, Vivado, Zynq, and other designated brands included herein are trademarks of Xilinx in the United States and other countries. AMBA, AMBA Designer, Arm, ARM1176JZ–S, CoreSight, Cortex, PrimeCell, Mali, and MPCore are trademarks of Arm Limited in the EU and other countries. PCI, PCIe, and PCI Express are trademarks of PCI–SIG and used under license. All other trademarks are the property of their respective owners.